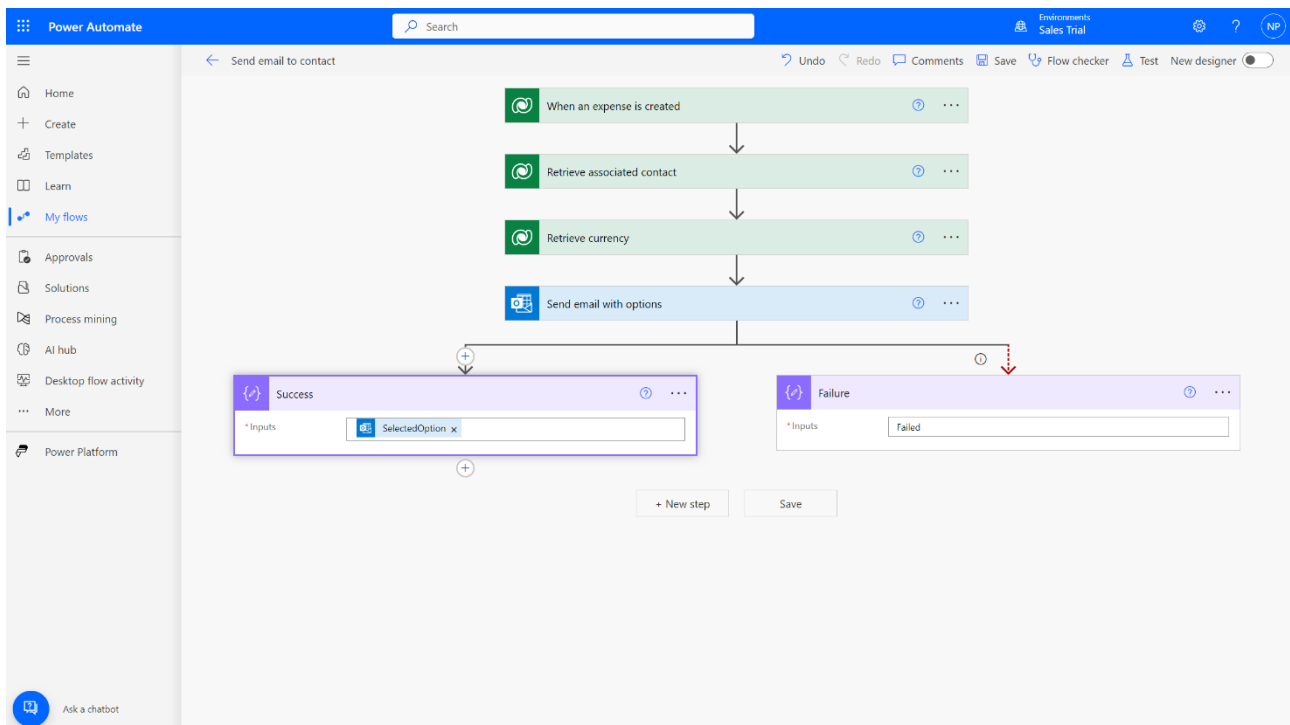


Creation of complex flows in Power Automate

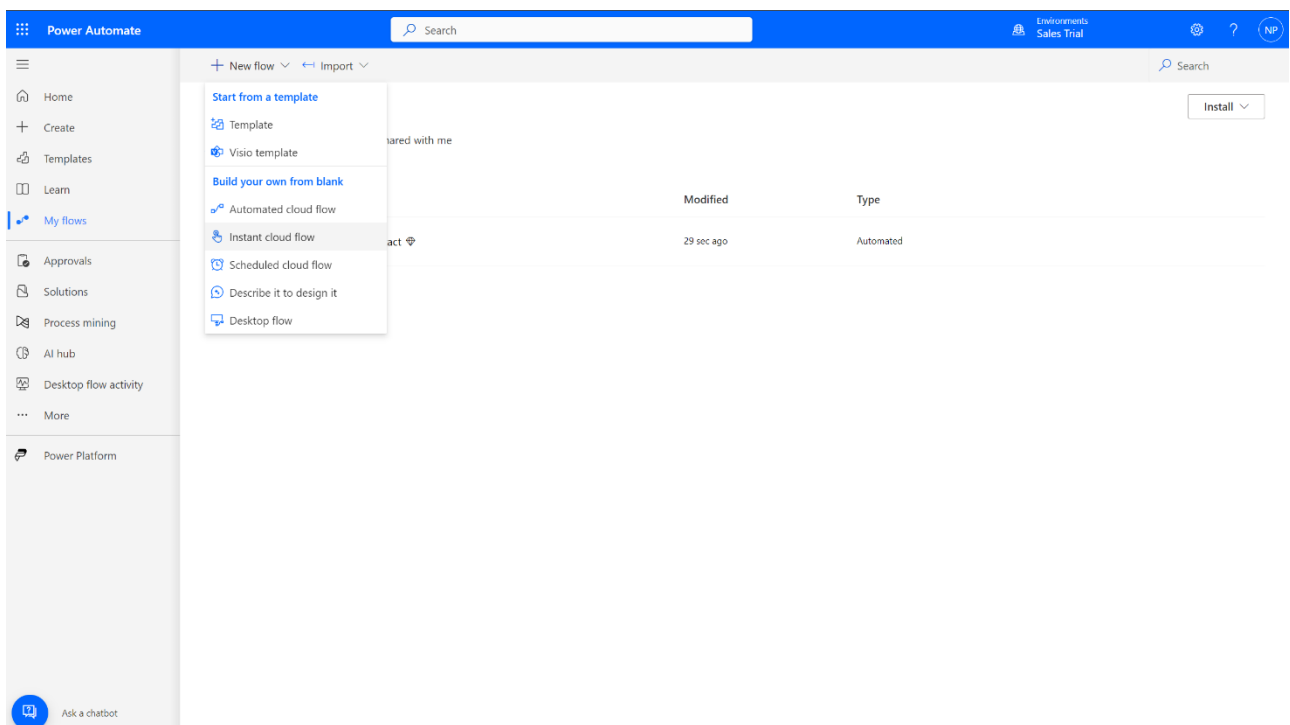
In the last lecture we focused on creating real-time and background processes. Then we have created the following automated cloud flow.



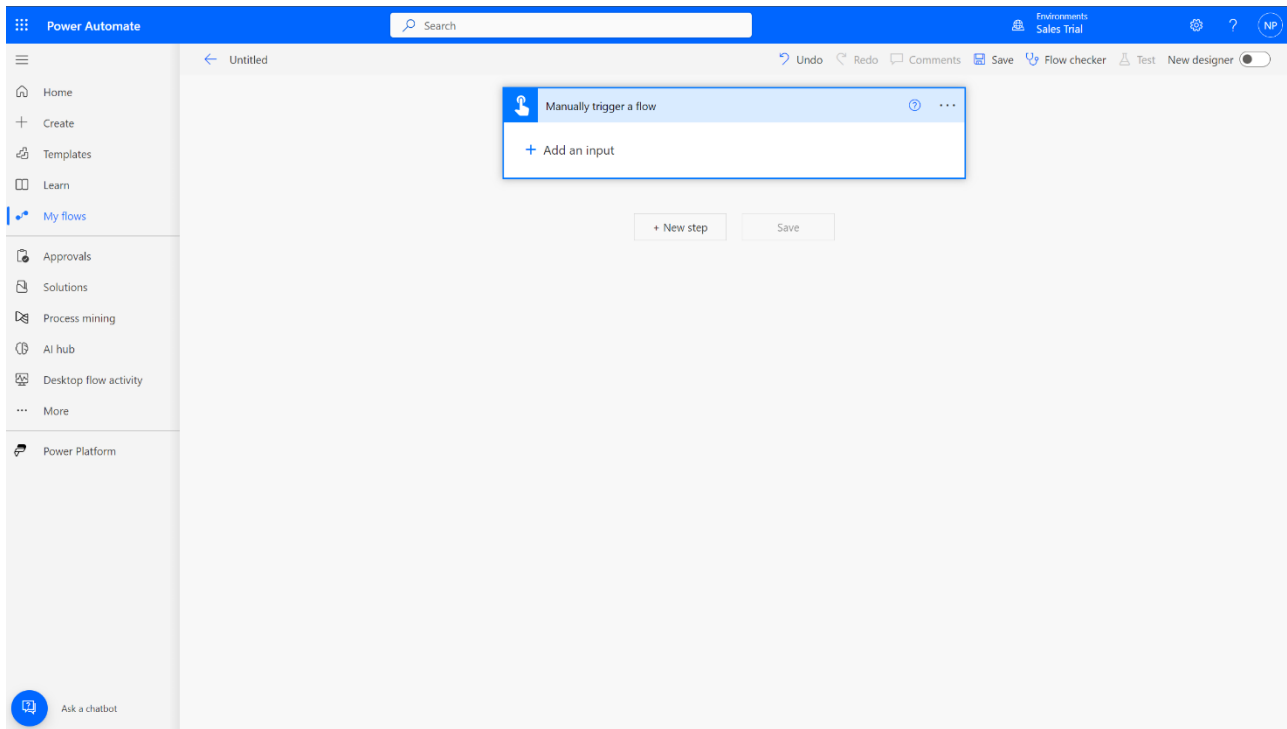
Basically, when a new expense is created in the database we send an email with options to the contact the expense belongs to.

1. Setting up filters and FetchXML queries

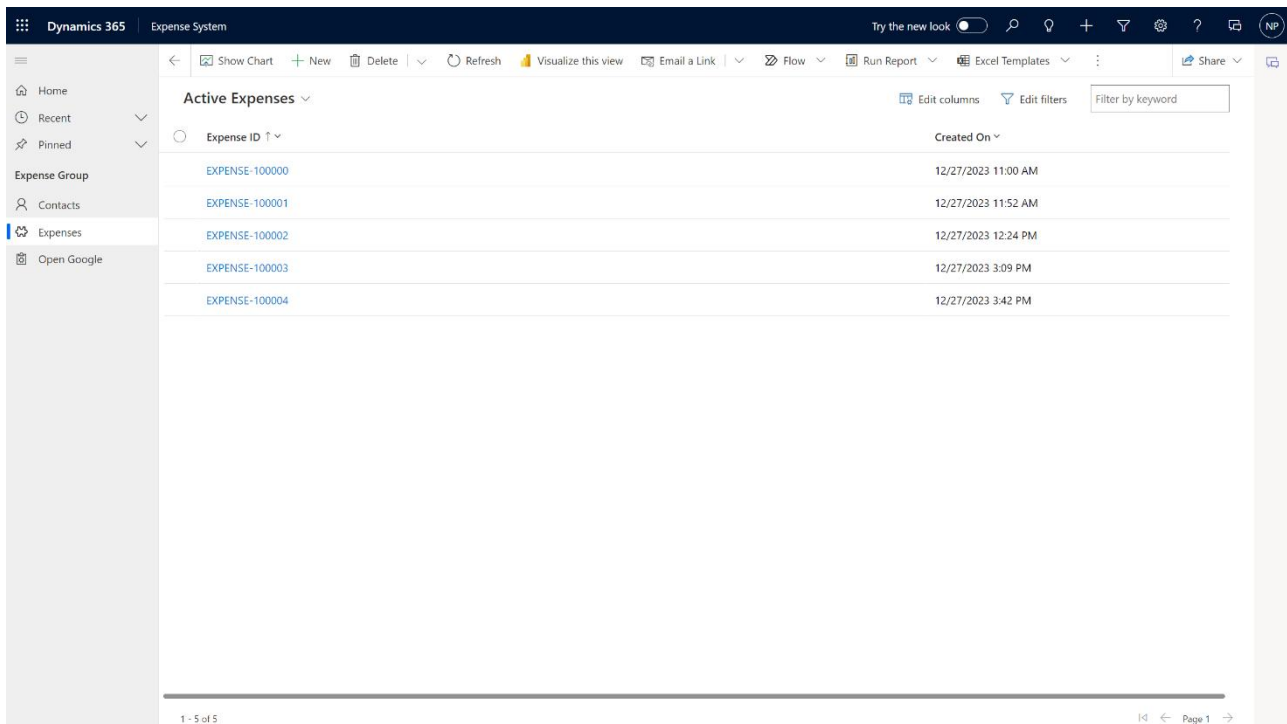
Let us create our first instant cloud flow.



Such a type of flow is not related to any trigger; instead, we have to run it manually. When creating a new untitled instant cloud flow, select the trigger **Manually trigger a flow**.



Then we create a new step selecting Microsoft Dataverse as connector, and List rows as action. Our aim is to retrieve a list of the expenses we have in Dynamics 365.



Here we have five expenses, which is a low number. In real-world scenarios, however, we might have thousands of records and, in general, we could be interested only in a subset of expense records. Moreover, not all the columns of the table we are calling are necessary.

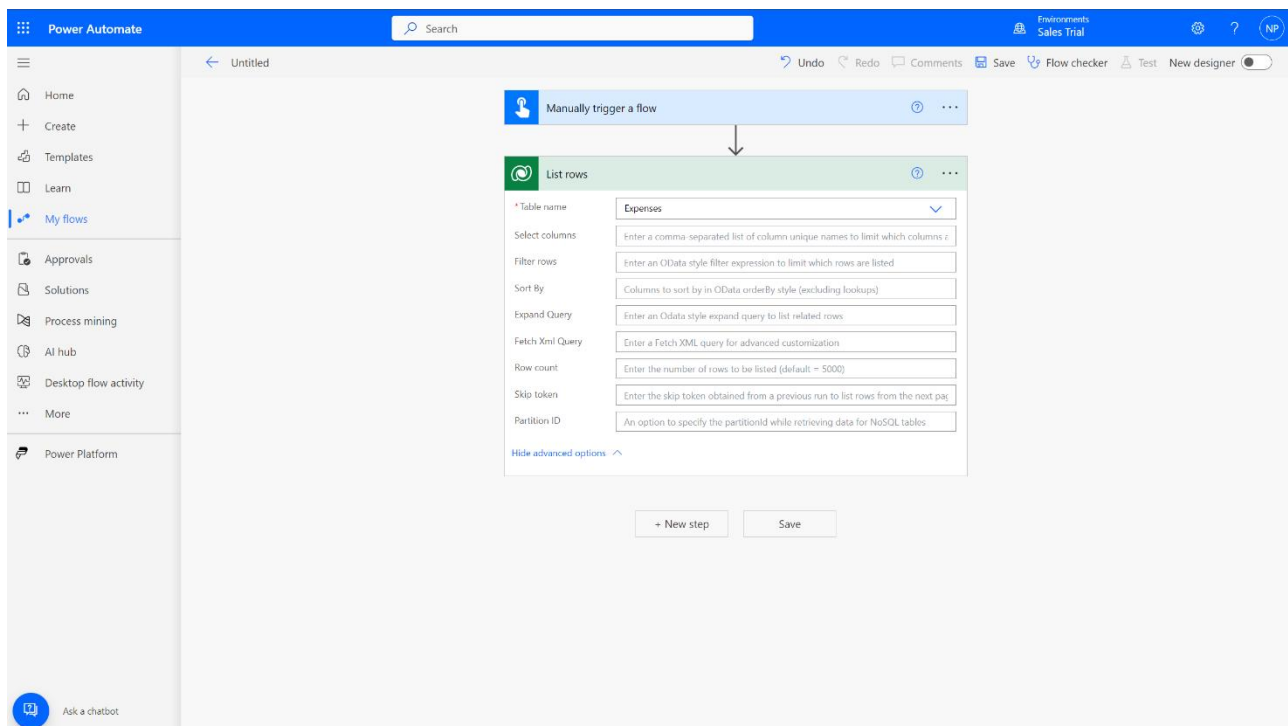
Using the logical names of the fields we have in Dynamics 365, we can select the columns we need and set appropriate filters in the operation of listing the rows of Expenses. Using Power Pane in Schema Name Copy Mode (to copy the logical names of the fields), we can build filters such as:

```
demo_name eq 'EXPENSE-100000' && demo_expenseamount ge 500
```

or maybe:

```
demo_name eq 'EXPENSE-100000' || demo_expenseamount 'EXPENSE-100001'
```

Another thing we can do is to define how to sort the columns and expand the query to show related rows.



When using the List rows action to retrieve data from a source, we can enhance our data retrieval by incorporating **FetchXML**. FetchXML is a query language specifically designed for Microsoft Dynamics 365 and CDS, enabling us to articulate complex criteria for data extraction. When configuring the List rows action, we first establish a connection to our data source, defining the authentication details. Subsequently, we select the entity or table from which you intend to retrieve data. Rather than relying solely on basic filters, FetchXML empowers us to craft sophisticated queries that precisely specify the conditions for data retrieval. We can either manually compose FetchXML queries or leverage tools like the FetchXML Builder in Dynamics 365 for a visual query creation experience. The FetchXML query we create outlines criteria, sorting, and other parameters relevant to our data needs. Once the FetchXML query is ready, we insert it into the appropriate field within the List rows action, informing the system about the specific data we are interested in. Following the execution of the List rows action, subsequent steps can then process the retrieved data. This may involve iterating through the results, applying conditional logic, or storing information in variables for further use.

Let us see how to set up and download a FetchXML file. Open Dynamics 365 and click on the filter icon.

The screenshot shows the Dynamics 365 Expense System interface. The left sidebar contains navigation options: Home, Recent, Pinned, Expense Group, Contacts, Expenses (selected), and Open Google. The main content area displays the details for an expense record with ID EXPENSE-100000. The record is saved and has tabs for General and Related. The General tab is active, showing fields for Expense ID, Expense Type (Flight), Expense Amount, Contact, Status Reason (Active), and Owner (Nicola Paracone (Busy)). A Description field is also present with a placeholder '---'.

We select Expenses and click Continue. A window in which we can define the desired filters will appear.

The screenshot shows the Dynamics 365 Expense System interface with the 'Edit filters: Expenses' dialog box open. The left sidebar is the same as in the previous screenshot. The main content area shows a list of 'Active Expenses' with IDs EXPENSE-100000 through EXPENSE-100004. The 'Edit filters: Expenses' dialog box is overlaid on the right, showing options to 'Reset to default', 'Download FetchXML', and 'Delete all filters'. It also indicates 'Showing live data' and provides a link to 'Change to retained data'. The filter configuration area shows a filter for 'Status' equals 'Active'. An 'Add' button is visible below the filter configuration. At the bottom right of the dialog are 'Apply' and 'Cancel' buttons.

In the following example we are retrieving the expenses which active and have Id equal to EXPENSE-100000.

The screenshot shows the Dynamics 365 Expense System interface. On the left, the navigation pane includes Home, Recent, Pinned, Expense Group, Contacts, Expenses (selected), and Open Google. The main area displays a list of 'Active Expenses' with the following entries:

Expense ID
EXPENSE-100000
EXPENSE-100001
EXPENSE-100002
EXPENSE-100003
EXPENSE-100004

At the bottom of the list, it shows '1 - 5 of 5'. On the right, the 'Edit filters: Expenses' dialog is open. It shows two filters connected by an AND operator:

- Status Equals Active
- Expense ID Equals EXPENSE-100000

Buttons for 'Reset to default', 'Download FetchXML', and 'Delete all filters' are visible. At the bottom right of the dialog are 'Apply' and 'Cancel' buttons.

Click Apply and then on Edit filters. Next, click on Download FetchXML.

The screenshot shows the Dynamics 365 Expense System interface. On the left, the navigation pane is the same as in the previous screenshot. The main area displays a list of 'Active Expenses*' with the following entry:

Expense ID
EXPENSE-001010

At the bottom of the list, it shows '1 - 1 of 1'. On the right, the 'Edit filters: Expenses' dialog is open. It shows two filters connected by an AND operator:

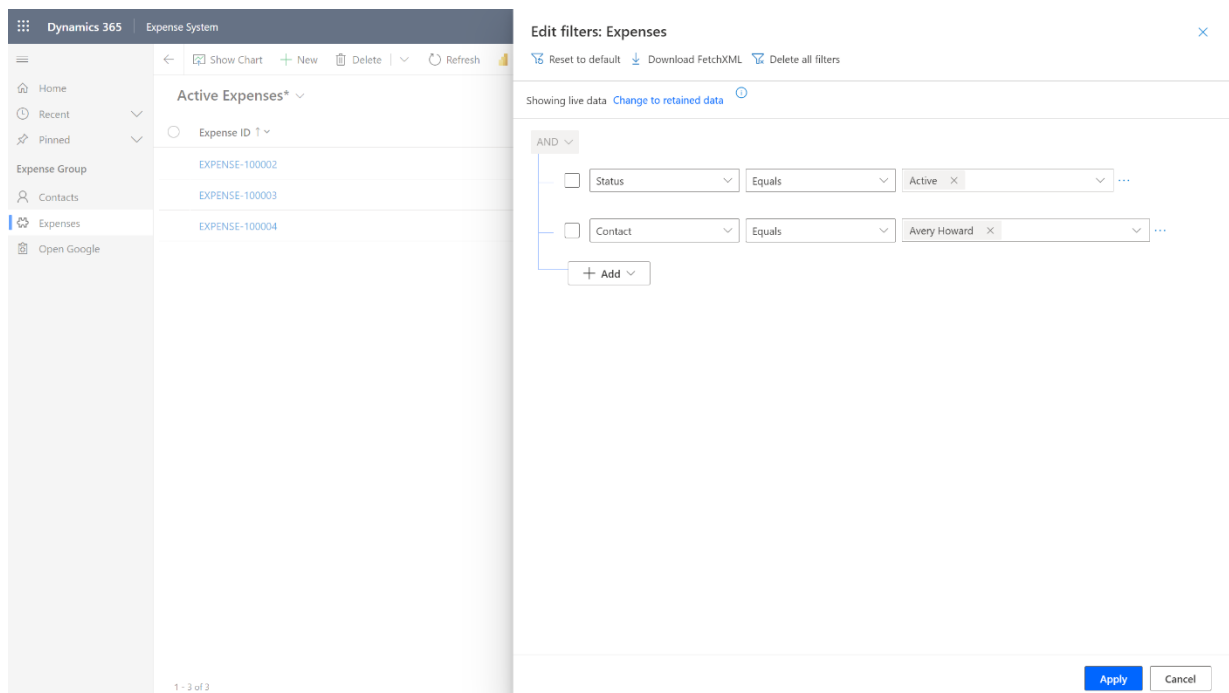
- Status Equals Active
- Expense ID Equals EXPENSE-001010

Buttons for 'Reset to default', 'Download FetchXML', and 'Delete all filters' are visible. At the bottom right of the dialog are 'Apply' and 'Cancel' buttons.

Proceeding this way, however, will download an XML file with a horrible indentation.

```
<fetch version="1.0" mapping="logical" savedqueryid="0720d2c3-b1f8-4ecc-ace6-ac1bf0303a8d" no-lock="false" distinct="true"><entity name="demo_expenses"><attribute name="demo_expensesid"/>
```

In a more realistic situation, it makes sense to filter the expenses by contact rather than by Id.

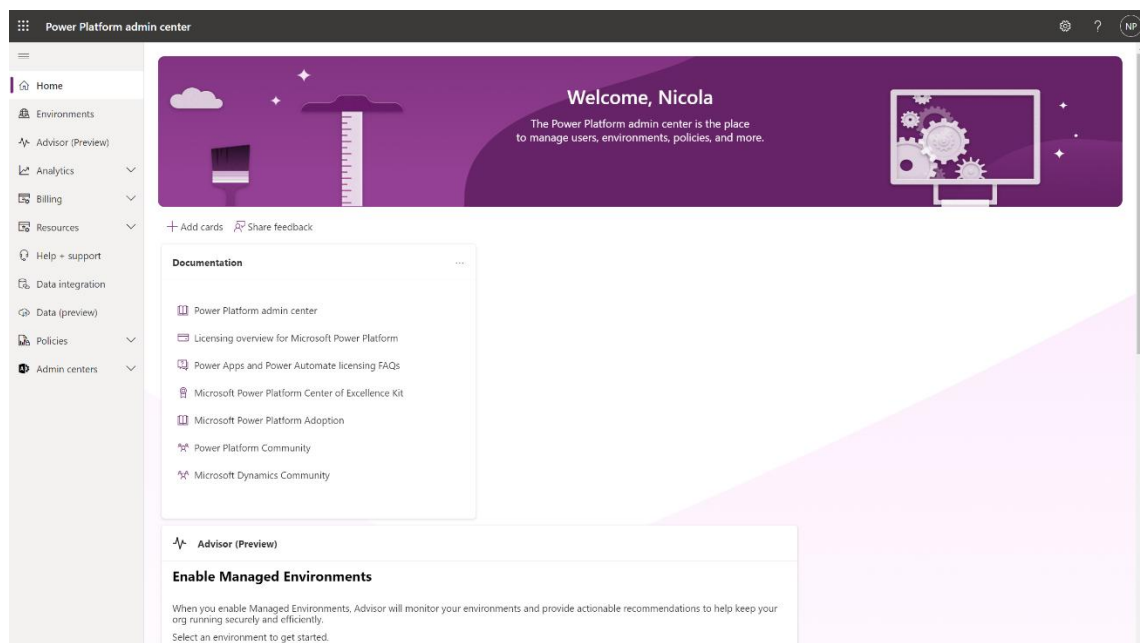


The following is the XML file we would download in this case (already properly formatted).

```
<fetch version="1.0" mapping="logical" savedqueryid="0720d2c3-b1f8-4ecc-ace6-ac1bf0303a8d" no-lock="false" distinct="true">
  <entity name="demo_expenses">
    <attribute name="demo_expensesid"/>
    <attribute name="demo_name"/>
    <attribute name="createdon"/>
    <order attribute="demo_name" descending="false"/>
    <filter type="and"><condition attribute="statecode" operator="eq" value="0"/>
    <condition attribute="demo_contact" operator="eq" value="{79ae8582-84bb-ea11-a812-000d3a8b3ec6}" uiname="Avery Howard" uitype="contact"/>
    </filter>
  </entity>
</fetch>
```

A workaround to the issue of having a bad FetchXML file is the described in the following.

If we go to admin.powerplatform.com, we access the admin center in which all the customizations to our organization take place.



Click on Environments and select Sales Trial.

The screenshot shows the 'Power Platform admin center' interface. The left sidebar contains navigation links: Home, Environments, Advisor (Preview), Analytics, Billing, Resources, Help + support, Data integration, Data (preview), Policies, and Admin centers. The main content area is titled 'Environments > Sales Trial'. It features several sections: 'Details' with fields like Environment URL, Region, Type, Organization ID, State, Refresh cadence, Security group, and Environment ID; 'Auditing' with options for enabling auditing and retaining logs; 'Version' showing the Dataverse version; 'Updates' for the 2023 release wave 2; 'Access' with links to security roles, teams, users, S2S apps, and business units; 'Resources' listing Dynamics 365 apps, Power Pages sites, Power Apps, and Flows; and 'Recent operations' with a full history link. A 'Generative AI features' section with an edit link is also visible at the bottom right.

Then click on Settings. This gives us a lot of options and features.

The screenshot shows the 'Power Platform admin center' interface with the 'Settings' page selected for the 'Sales Trial' environment. The left sidebar is the same as in the previous screenshot. The main content area is titled 'Environments > Sales Trial > Settings'. It includes a search bar for settings and a list of categories with expandable sub-items: 'Product' (Behavior, Collaboration, Features, Languages, Privacy + Security), 'Business' (Business closures, Calendar, Connection roles, Currencies), 'Users + permissions' (Application users, Business units (Preview), Column security profiles, Hierarchy security), 'Audit and logs' (Audit log management, Audit settings, Audit summary view, Entity and field audit settings), 'Templates' (Access team templates, Article templates, Contract templates, Data import templates), and 'Updates' (App update settings (Preview)). On the right side, there are expandable sections for 'Email' (Email settings, Email tracking, Mailboxes, Server profiles), 'Integration' (Document management settings, Synchronization, Teams integration settings, Yammer), 'Data management' (Auto numbering, Automatic record creation policies, Bulk deletion, Data import wizard), 'Encryption' (Data encryption), and 'Resources' (All legacy settings, Dynamics 365 App for Outlook).

Under Product we have Features.

The screenshot shows the 'Features' page in the Power Platform admin center. The left sidebar contains navigation links: Home, Environments, Advisor (Preview), Analytics, Billing, Resources, Help + support, Data integration, Data (preview), Policies, and Admin centers. The main content area is titled 'Environments > Sales Trial > Settings > Features' and includes a link to 'Learn more about Features'. The features are organized into sections:

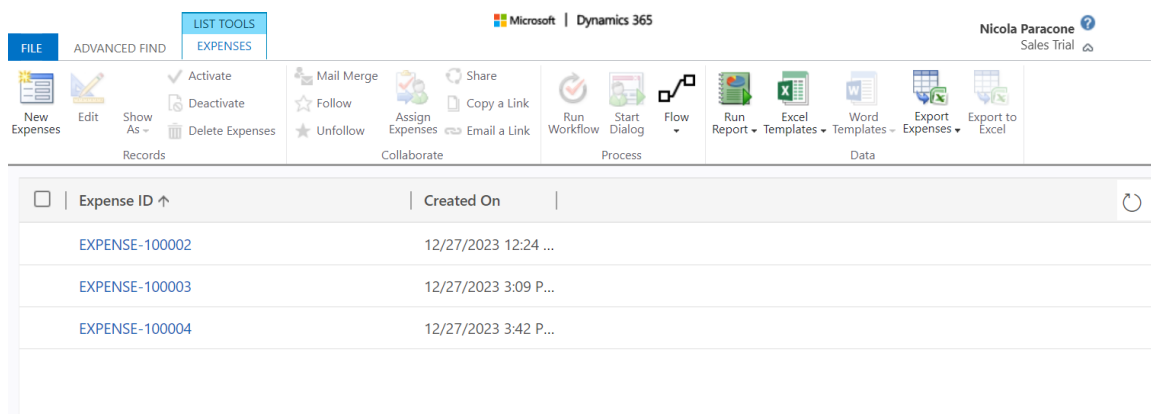
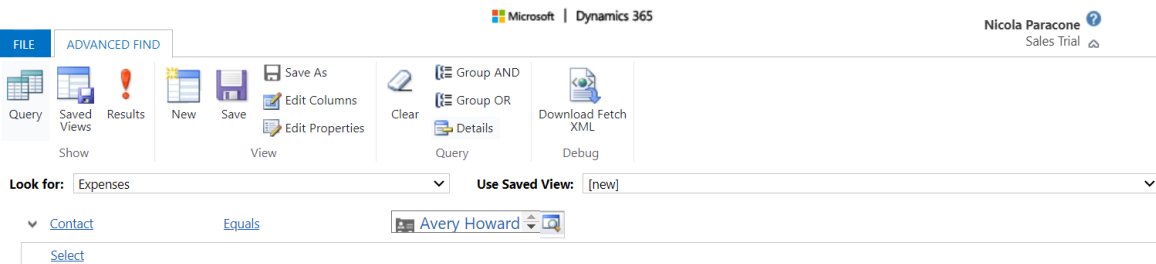
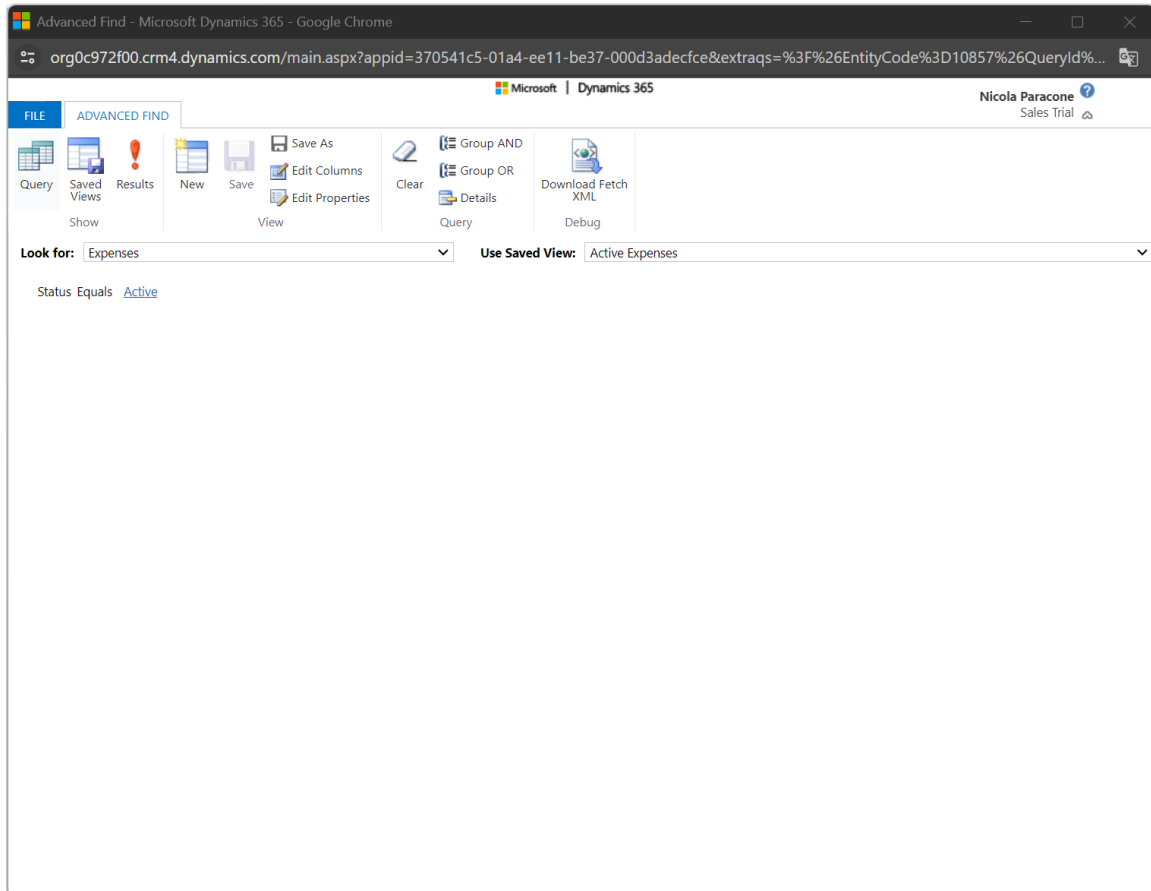
- Copilot** (Preview): Allow canvas editors to get AI-powered answers to how-to questions and AI Builder GPT experiences. Currently in preview. **On**
- AI Builder**: Enable the usage of model types that are in preview. **AI Builder preview models**: **On**
- Embedded content**:
 - Power BI visualization embedding: **Off**
 - Bing Maps: **Off**
 - Prevent social data in Dynamics: **Off**
- Communications**:
 - Country/region code prefixing for numbers: **On**
 - Set the telephony provider:
 - Use Skype: **Selected**
 - Use Skype for Business: **Off**
- Hosted RPA**
- Search**: Manage how search works in this environment. **Dataverse search**: **Off**. **Share search analytics with Microsoft**: **Off**. **Single table search option**: **Off**. **Prevent slow keyword filter for quick find terms on view pages**: **Off**. **Quick Find record limits**: **On**. **Use quick find view of an entity for searching on grids and sub-grids**: **Off**
- Map and address services**: **Limited**

Under Advanced find options turn off Modern advanced find in model-driven apps and save.

The screenshot shows the 'Advanced find options' page in the Power Platform admin center. The left sidebar is the same as the previous screenshot. The main content area is titled 'Environments > Sales Trial > Settings > Advanced find options' and includes a link to 'Learn more about Advanced find options'. The settings are organized into sections:

- Grids and views**:
 - Enable the modern read-only grid experience: **On**
 - Show the Edit columns button on views: **On**
 - Show the Edit filters button on views: **On**
- Advanced find options**: These options let users create, manage, and search views inside model-driven apps. **Modern advanced find in model-driven apps**: **Off**. When on, legacy advanced find will be replaced with a modern experience with enhanced view management capabilities. **Allow users to hide system views**: **Off**
- Collaboration**: Collaboration configuration has moved to Collaboration settings
- Activities**: Show only activities configured in the app when accessing "New activity" button in related activities of a record. Setting to "off" will show all activities. **On**
- Asynchronous merge**: When enabled, records merged through the Unified Client interface will be merged asynchronously in the background. **Allow records to be merged asynchronously**: **Off**
- Create new canvas apps and cloud flows in Dataverse solutions (Preview)**
- Delete disabled users**: Show the "Delete user" option in "Settings > Users" for disabled users, so you can permanently delete them from this environment. **Enable delete disabled users**: **Off**
- TDS endpoint**: Enables connecting from Power BI using Tabular Data Stream(TDS) Protocol. **Enable TDS endpoint**: **On**
- Record ownership across Business Units**: This feature can take several minutes to enable. **Record ownership across Business Units**: **Off**
- Dynamics 365 Sales mobile app**: Set up the Dynamics 365 Sales mobile app. These settings will apply to all users in this environment. **Create new records and activities using Unified Interface forms instead of quick view forms**: **Off**. **Disable the option to quick create new records and activities in the Mobile application**: **Off**. **Edit and view records and activities using Unified Interface forms instead of quick view forms**: **Off**
- Power Apps Ideas for Canvas Apps**: Share ideas query data to help improve ideas results. Requires agreement to the terms of service. **Off**

If we now refresh the previous window in Dynamics 365 (in which we filtered expenses by contact) and select the Advanced find icon again, the classic editor will pop up. Advanced find is used when we want to retrieve some data by applying our own filters.



Then we can download the FetchXML file (old version), which will have the following appearance.

```
<fetch version="1.0" output-format="xml-platform" mapping="logical" distinct="false">
  <entity name="demo_expenses">
    <attribute name="demo_expensesid" />
    <attribute name="demo_name" />
    <attribute name="createdon" />
    <order attribute="demo_name" descending="false" />
    <filter type="and">
      <condition attribute="demo_contact" operator="eq" uiname="Avery Howard" uitype="contact" value="{79AE8582-848B-EA11-A812-000D3A8B3EC6}" />
    </filter>
  </entity>
</fetch>
```

Notice the evident structural differences between this XML file and the previous one.

Copy the XML file content into the Fetch Xml Query field of the List rows block and add a Compose to store the temporary data.

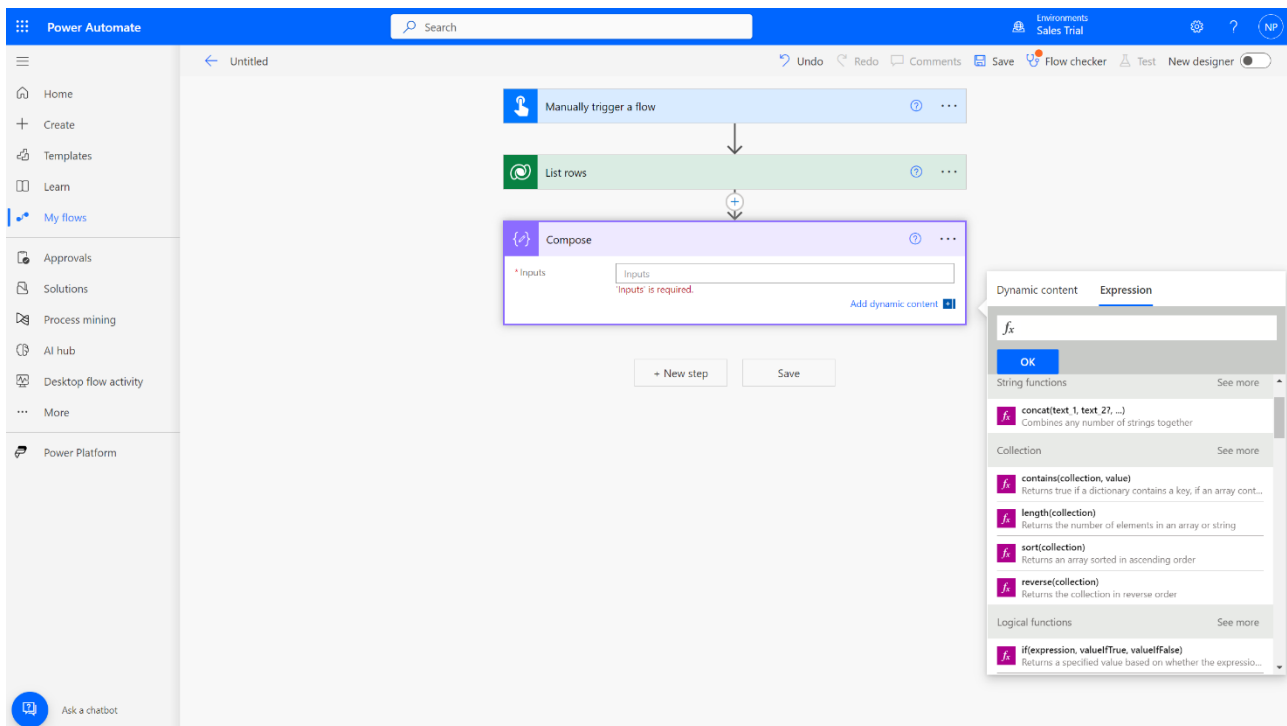
The screenshot displays the Power Automate workflow editor. At the top, a 'Manually trigger a flow' block is connected to a 'List rows' block. The 'List rows' block is configured with the following settings:

- Table name: Expenses
- Select columns: Enter a comma-separated list of column unique names to limit which columns are returned.
- Filter rows: Enter an OData style filter expression to limit which rows are listed.
- Sort By: Columns to sort by in OData orderBy style (excluding lookups).
- Expand Query: Enter an Odata style expand query to list related rows.
- Fetch Xml Query:

```
<fetch version="1.0" output-format="xml-platform" mapping="logical" distinct="false">
  <entity name="demo_expenses">
    <attribute name="demo_expensesid" />
    <attribute name="demo_name" />
    <attribute name="createdon" />
    <order attribute="demo_name" descending="false" />
    <filter type="and">
      <condition attribute="demo_contact" operator="eq" uiname="Avery Howard" uitype="contact" value="{79AE8582-848B-EA11-A812-000D3A8B3EC6}" />
    </filter>
  </entity>
</fetch>
```
- Row count: Enter the number of rows to be listed (default = 5000).
- Skip token: Enter the skip token obtained from a previous run to list rows from the next page.
- Partition ID: An option to specify the partitionId while retrieving data for NoSQL tables.

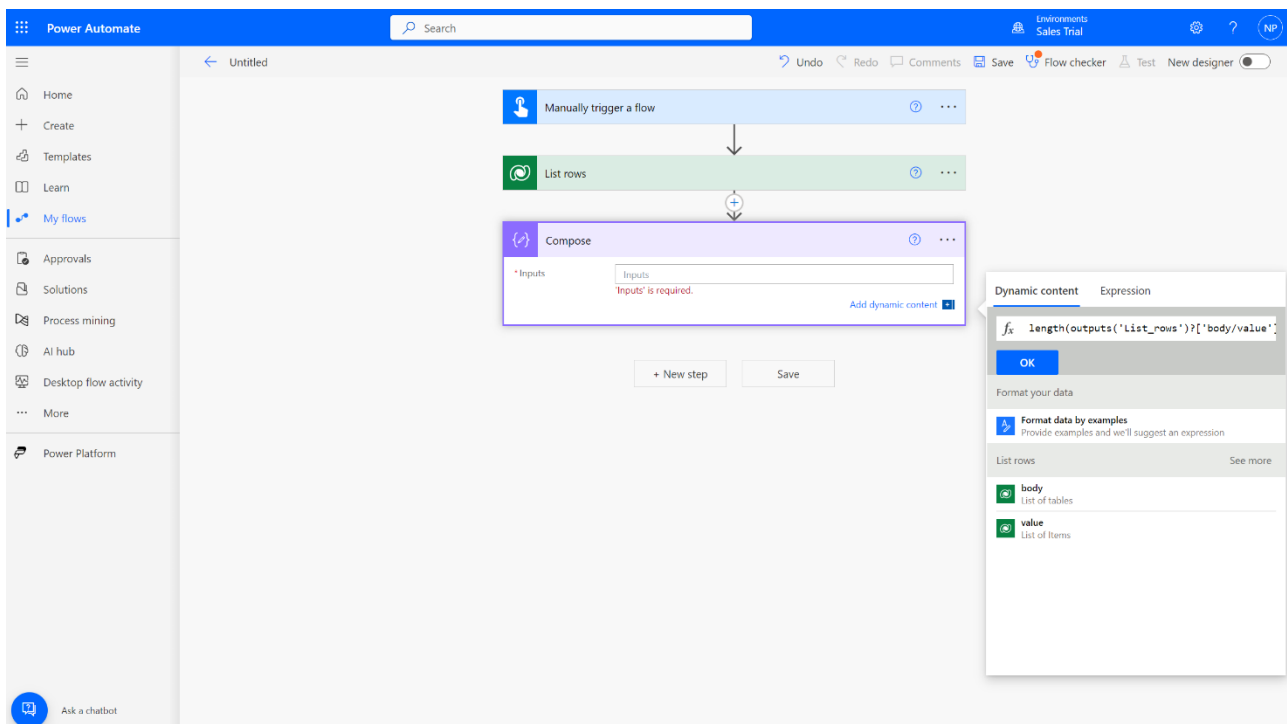
Below the 'List rows' block, a 'Compose' block is added. The 'Compose' block has an 'Inputs' field with the text 'Inputs' and a red error message: 'Inputs is required.' To the right of the 'Compose' block, a 'Dynamic content' pane is open, showing a search bar and a list of dynamic content options under the 'List rows' category:

- value: List of items
- body: List of tables
- body/value - Item
- Next link: The url to fetch next page data.



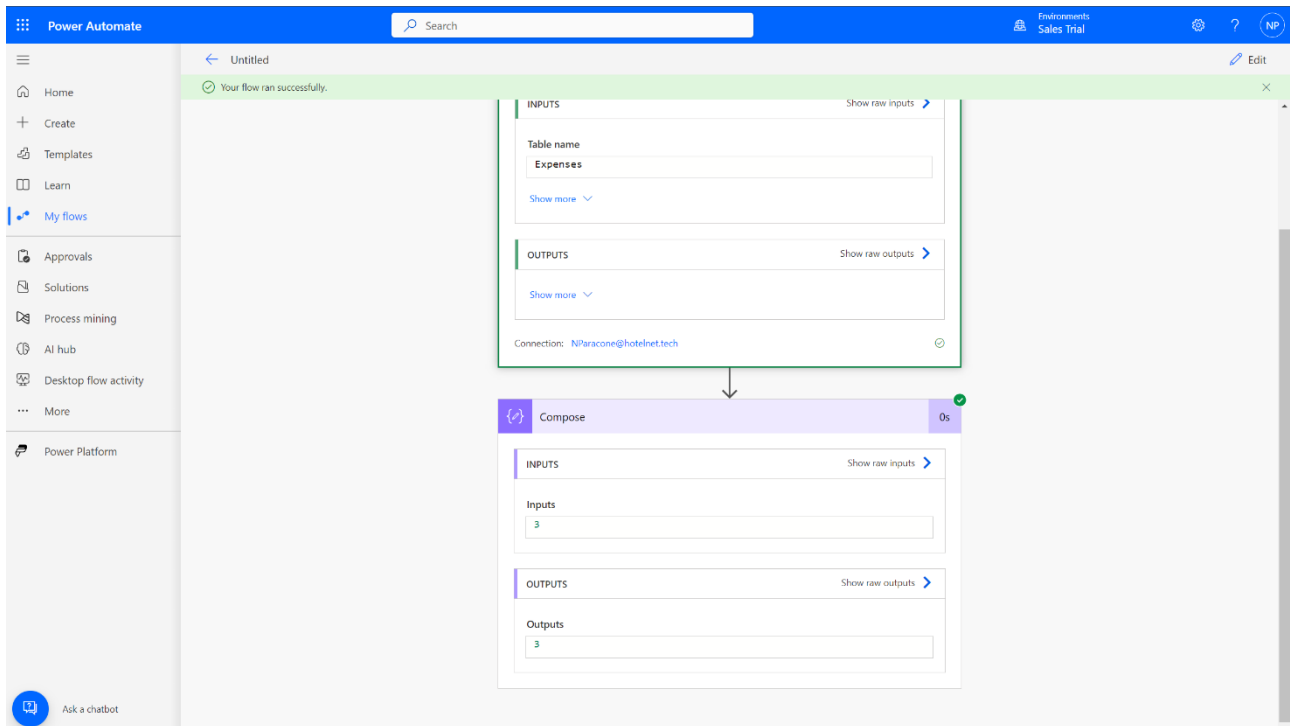
For `contains(collection,value)` we pass a collection of records and check if it contains some custom value. Other useful expressions are `length(collection)`, `sort(collection)` and `reverse (collection)`.

We use `length` and `set` as collection value.

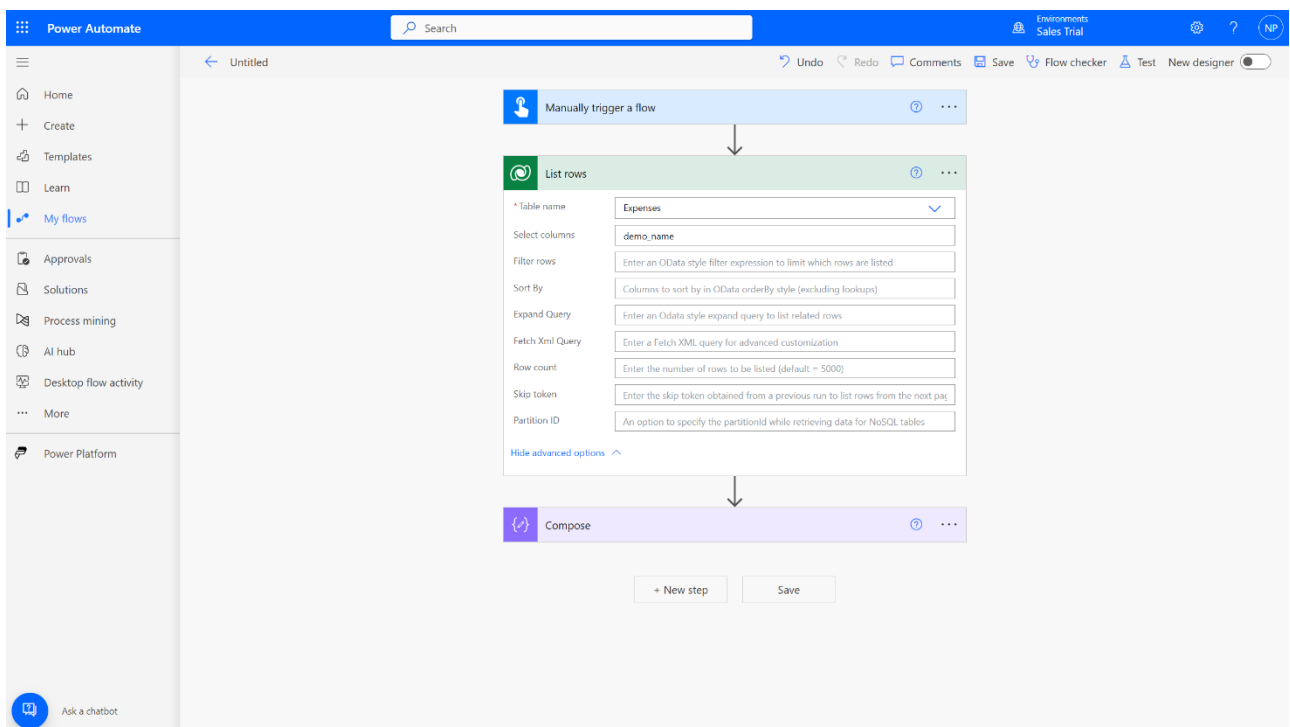


At this point, we save and test the flow.

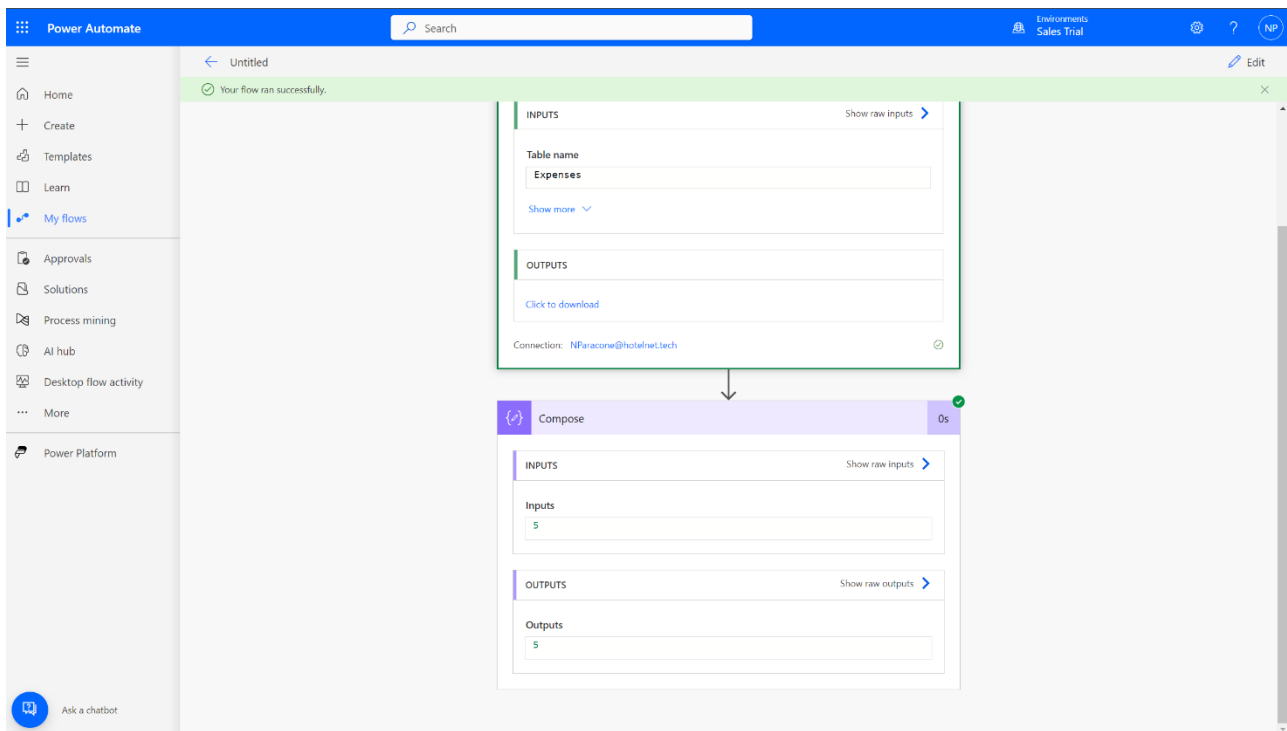
If everything worked fine, the following should be the result displayed (we have in fact only three expenses associated with Avery Howard).



Now modify the List rows block as shown.



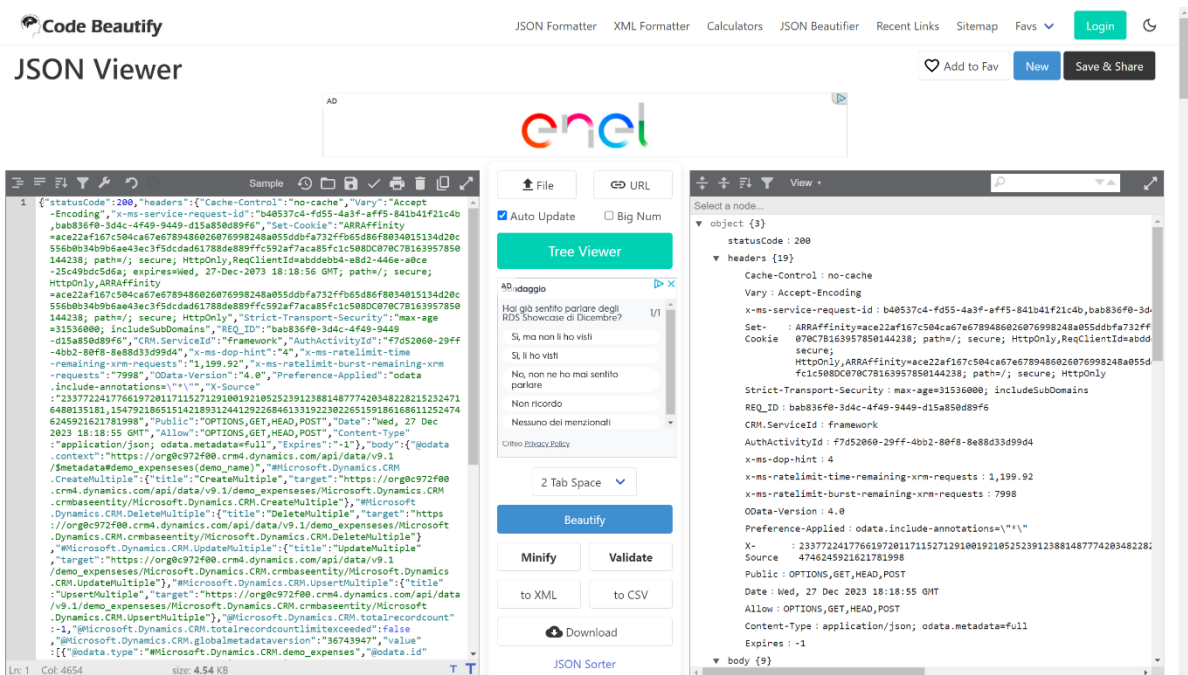
If we have five expense requests, the output result of testing will be 5; in fact, we are only retrieving the names.



Click on Click to download.

```
({"statusCode":200,"headers":{"Cache-Control":"no-cache","Vary":"Accept-Encoding","x-ms-service-request-id":"b48537c4-fd55-4a3f-af95-841b4f21c4b,bab836f0-3d4c-4f49-9449-d15a850d89f6","Set-Cookie":"ARRAffinity=ace22af167c504ca67e678948602676998248a055dbfa732f7f65d86f8034015134d20c556b0b34b9b5ae43ec3f5dcad61788de889ffc592af7aca85f1c580c078c78163957850144238; path=/; secure; HttpOnly,ReqClientId=abddbb4-e8d2-446e-a8ce-25c49bdc5d6a; expires=Wed, 27-Dec-2023 18:18:56 GMT; path=/; secure; HttpOnly,ARRAffinity=ace22af167c504ca67e678948602676998248a055dbfa732f7f65d86f8034015134d20c556b0b34b9b5ae43ec3f5dcad61788de889ffc592af7aca85f1c580c078c78163957850144238; path=/; secure; HttpOnly,\"Strict-Transport-Security\":\"max-age=31536000; includeSubDomains\"},\"REQ_ID\":\"bab836f0-3d4c-4f49-9449-d15a850d89f6\",\"CRM.ServiceId\":\"Framework\",\"AuthActivityId\":\"f7d52060-29ff-4bb2-80f8-8e88d33d9904\",\"x-ms-dop-hint\":\"4\",\"x-ms-ratelimit-time-remaining-xrm-requests\":\"1,199.92\",\"x-ms-ratelimit-burst-remaining-xrm-requests\":\"7998\",\"odata-version\":\"4.0\",\"Preference-Applied\":\"odata.include-annotations=\\\"*\\\"\",\"x-ms-ratelimit-time-remaining-xrm-requests\":\"7998\"},\"odata-version\":\"4.0\",\"Preference-Applied\":\"odata.include-annotations=\\\"*\\\"\",\"Public\":\"OPTIONS,GET,HEAD,POST\",\"Date\":\"Wed, 27 Dec 2023 18:18:55 GMT\",\"Allow\":\"OPTIONS,GET,HEAD,POST\",\"Content-Type\":\"application/json;odata.metadata=full\",\"Expires\":\"-1\"},\"body\":{\"@odata.context\":\"https://orgc972f00.crm.dynamics.com/api/data/v9.1/$metadata#Microsoft.Dynamics.CRM.expenses\",\"@odata.type\":\"Microsoft.Dynamics.CRM.expenses\",\"demo_name\":\"Microsoft.Dynamics.CRM.CreateMultiple\"},{\"title\":\"DeleteMultiple\",\"target\":\"https://orgc972f00.crm.dynamics.com/api/data/v9.1/demo_expenses/Microsoft.Dynamics.CRM.crmbaseentity/Microsoft.Dynamics.CRM.DeleteMultiple\"},{\"title\":\"UpdateMultiple\",\"target\":\"https://orgc972f00.crm.dynamics.com/api/data/v9.1/demo_expenses/Microsoft.Dynamics.CRM.crmbaseentity/Microsoft.Dynamics.CRM.UpdateMultiple\"},{\"title\":\"UpsertMultiple\",\"target\":\"https://orgc972f00.crm.dynamics.com/api/data/v9.1/demo_expenses/Microsoft.Dynamics.CRM.crmbaseentity/Microsoft.Dynamics.CRM.UpsertMultiple\"},{\"title\":\"Microsoft.Dynamics.CRM.totalrecordcountlimitexceeded\":false,\"Microsoft.Dynamics.CRM.globalmetadatasversion\":\"36743947\",\"value\":{\"@odata.type\":\"Microsoft.Dynamics.CRM.demo_expenses\",\"@odata.id\":\"https://orgc972f00.crm.dynamics.com/api/data/v9.1/demo_expenses(df9a3db2-9ea4-e011-be37-00003adecf3e)\",\"@odata.etag\":\"/M/\\\"36743735\\\"\",\"@odata.editLink\":\"demo_expenses(df9a3db2-9ea4-e011-be37-00003adecf3e)\",\"demo_name\":\"EXPENSE-100000\",\"demo_expensesid@odata.type\":\"@Guid\",\"demo_expensesid\":\"df9a3db2-9ea4-e011-be37-00003adecf3e\"},{\"@odata.type\":\"Microsoft.Dynamics.CRM.demo_expenses\",\"@odata.id\":\"https://orgc972f00.crm.dynamics.com/api/data/v9.1/demo_expenses(9db53083-aa4-e011-be37-00003adecf3e)\",\"@odata.etag\":\"/M/\\\"36742599\\\"\",\"@odata.editLink\":\"demo_expenses(9db53083-aa4-e011-be37-00003adecf3e)\",\"demo_name\":\"EXPENSE-100001\",\"demo_expensesid@odata.type\":\"@Guid\",\"demo_expensesid\":\"9db53083-aa4-e011-be37-00003adecf3e\"},{\"@odata.type\":\"Microsoft.Dynamics.CRM.demo_expenses\",\"@odata.id\":\"https://orgc972f00.crm.dynamics.com/api/data/v9.1/demo_expenses(9db53083-aa4-e011-be37-00003adecf3e)\",\"@odata.etag\":\"/M/\\\"36742896\\\"\",\"@odata.editLink\":\"demo_expenses(9db53083-aa4-e011-be37-00003adecf3e)\",\"demo_name\":\"EXPENSE-100002\",\"demo_expensesid@odata.type\":\"@Guid\",\"demo_expensesid\":\"9db53083-aa4-e011-be37-00003adecf3e\"},{\"@odata.type\":\"Microsoft.Dynamics.CRM.demo_expenses\",\"@odata.id\":\"https://orgc972f00.crm.dynamics.com/api/data/v9.1/demo_expenses(f7c1d96-c1a4-e011-be37-00003adecf3e)\",\"@odata.etag\":\"/M/\\\"36743994\\\"\",\"@odata.editLink\":\"demo_expenses(f7c1d96-c1a4-e011-be37-00003adecf3e)\",\"demo_name\":\"EXPENSE-100003\",\"demo_expensesid@odata.type\":\"@Guid\",\"demo_expensesid\":\"f7c1d96-c1a4-e011-be37-00003adecf3e\"},{\"@odata.type\":\"Microsoft.Dynamics.CRM.demo_expenses\",\"@odata.id\":\"https://orgc972f00.crm.dynamics.com/api/data/v9.1/demo_expenses(2a880b2a-c6a4-e011-be37-00003adecf3e)\",\"@odata.etag\":\"/M/\\\"36744057\\\"\",\"@odata.editLink\":\"demo_expenses(2a880b2a-c6a4-e011-be37-00003adecf3e)\",\"demo_name\":\"EXPENSE-100004\",\"demo_expensesid@odata.type\":\"@Guid\",\"demo_expensesid\":\"2a880b2a-c6a4-e011-be37-00003adecf3e\"}}]}
```

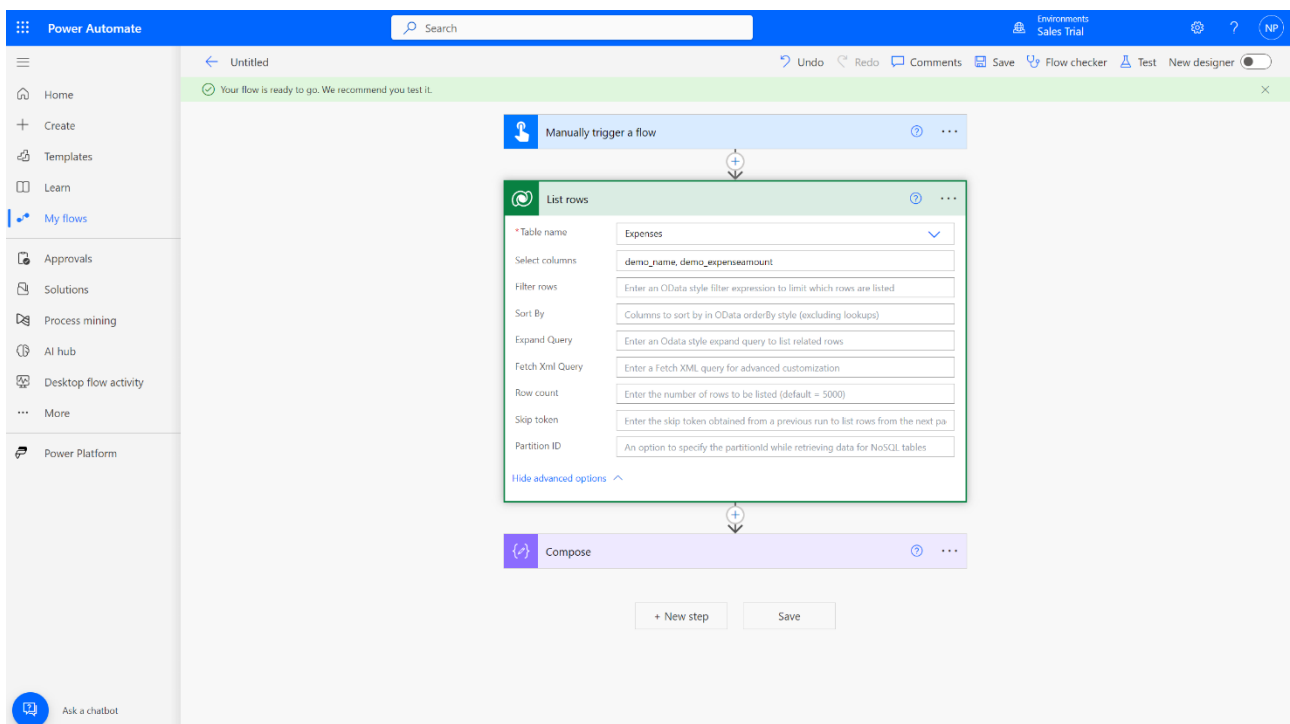
The data displayed are quite hard to read, but we can open a JSON beautifier to have a better view.



Once the JSON is beautified, we obtain a much clearer picture of the data of each expense that the flow has retrieved.



Let us add one more field, separated by comma.



The new beautified JSON file will be the following.

The screenshot shows the Code Beautify website's JSON Viewer. The interface includes a top navigation bar with links for JSON Formatter, XML Formatter, Calculators, JSON Beautifier, Recent Links, Sitemap, and a Login button. The main area is titled 'JSON Viewer' and features a 'Beautify' button, 'Minify', 'Validate', 'to XML', and 'to CSV' options. A 'Tree Viewer' on the left shows the JSON structure. The right pane displays the beautified JSON code, which includes fields like 'transactioncurrencyid', 'demo_name', 'demo_expenseamount', and 'demo_expenseamount@odata.type'.

This also gives the expense amount, and since it is related to the currency field, it also gives the currency options.

2. Setting ODATA filters to rows

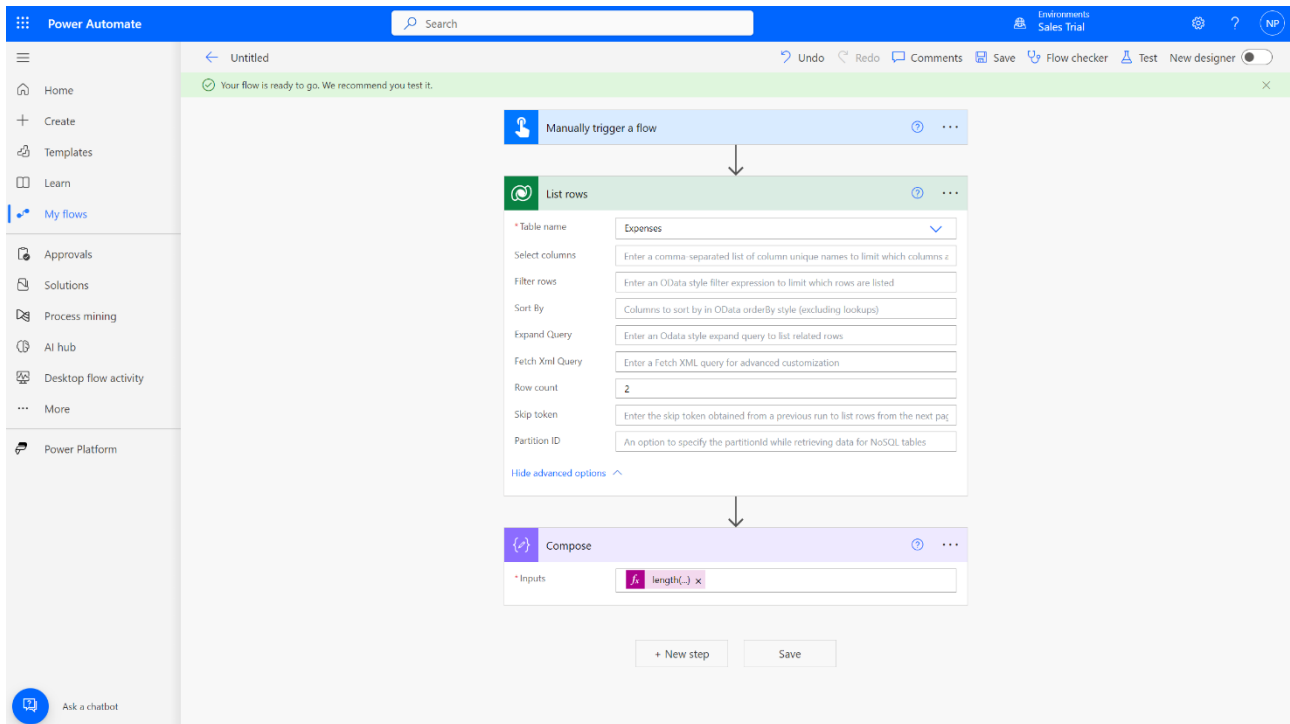
To set an ODATA (Open Data Protocol) filter on the row, we can write something like in the following figure.

The screenshot shows the Power Automate interface. The flow is titled 'Untitled'. It starts with a 'Manually trigger a flow' action. This is followed by a 'List rows' action, which is configured with the following settings: 'Table name' is 'Expenses', 'Filter rows' is 'demo_name eq EXPENSE-100000', 'Sort By' is 'Columns to sort by in OData order style (excluding lookups)', 'Expand Query' is 'Enter an Odata style expand query to list related rows', 'Fetch Xml Query' is 'Enter a Fetch XML query for advanced customization', 'Row count' is 'Enter the number of rows to be listed (default = 5000)', 'Skip token' is 'Enter the skip token obtained from a previous run to list rows from the next pag', and 'Partition ID' is 'An option to specify the partitionid while retrieving data for NuSQL tables'. The flow then proceeds to a 'Compose' action, which has an input of 'length(1)'. The flow is saved and ready to be tested.

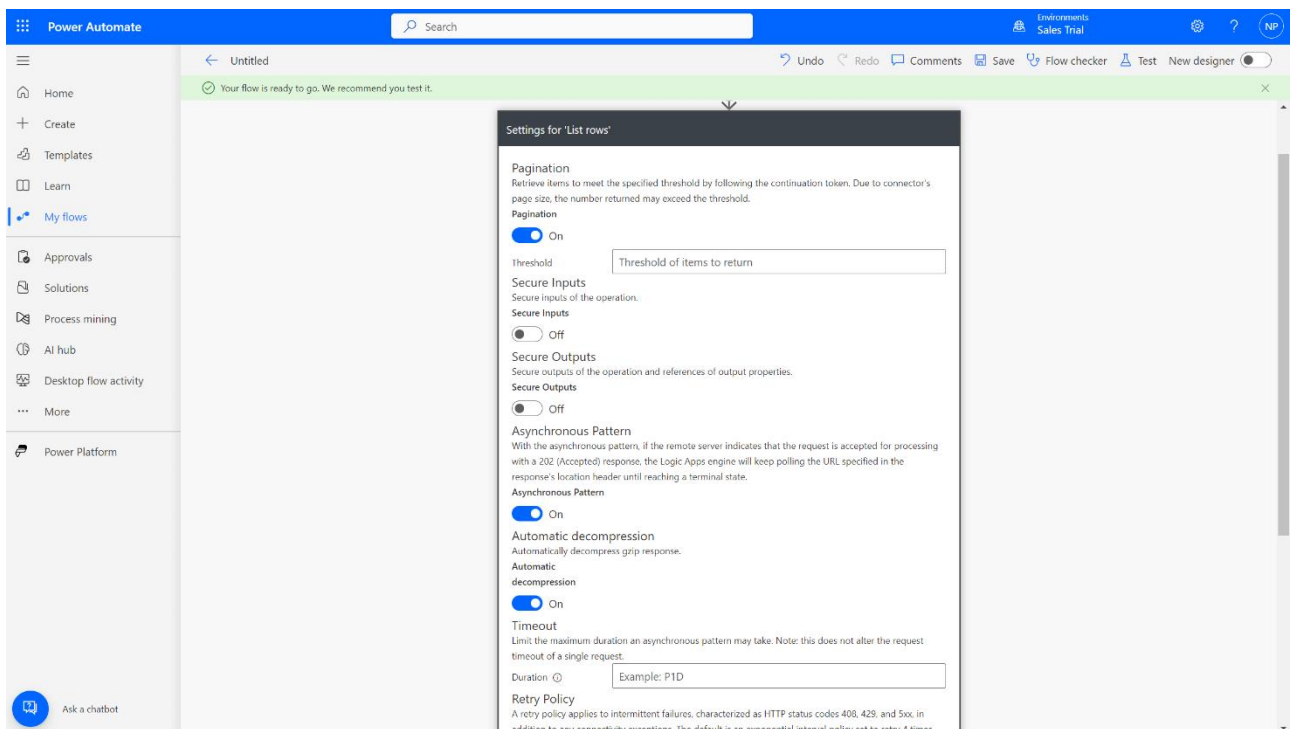
The output will be associated only with the expense with that precise name.

Another important thing is that List rows gives a maximum of 5000 records.

If we type Row count = 2, it will give us the top two records.

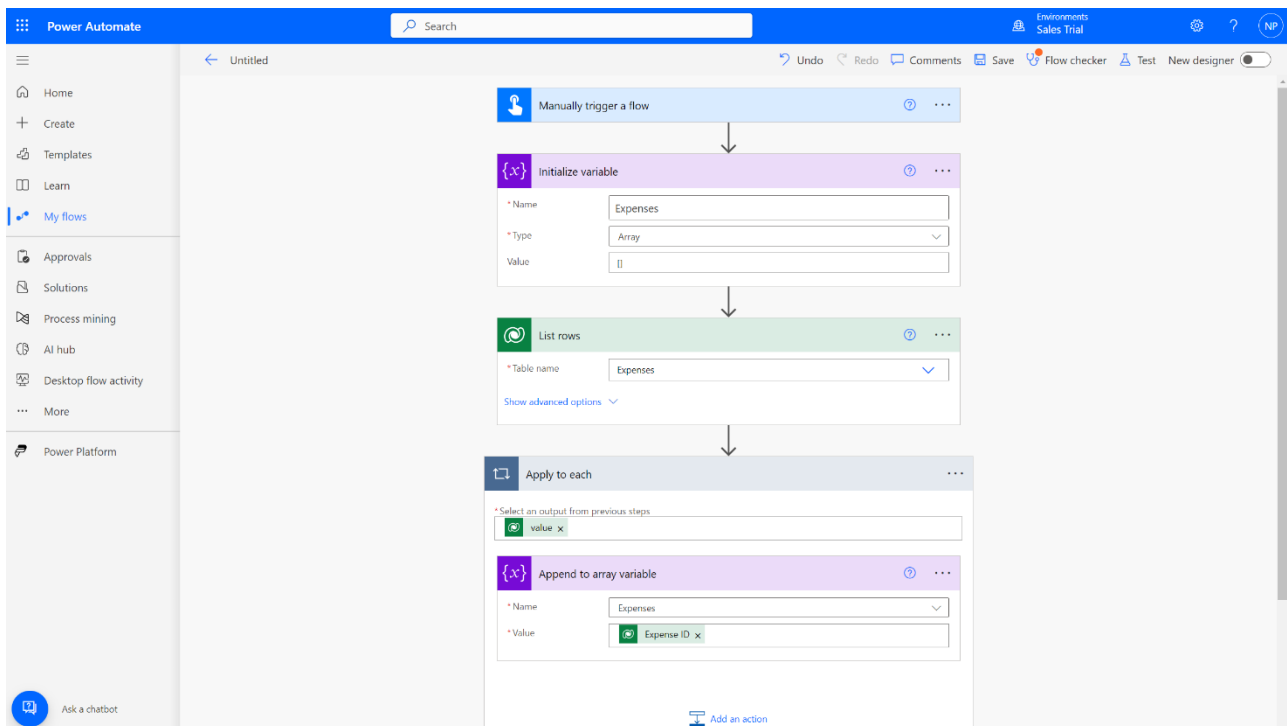


If we have several thousand records, however, how do we retrieve them all? We access and enable Pagination in the settings of the List rows block.



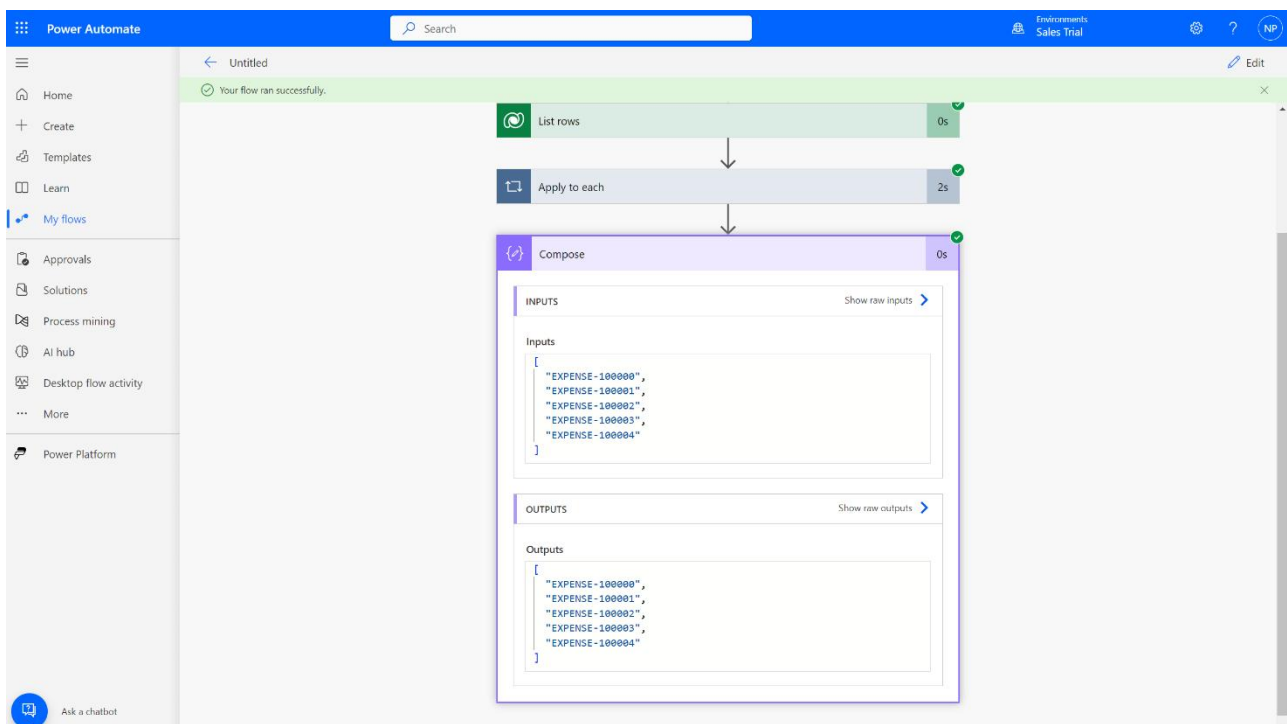
3. Variables and arrays

Another fundamental concept in Power Automate is that of **variable**. In the following flow we list all the expenses, add them to an array, create a table and send it via email.



After the List rows block, we add an Apply to each block containing the action Append to array variable. This operation is basically a for loop (or foreach) running on the elements of the list.

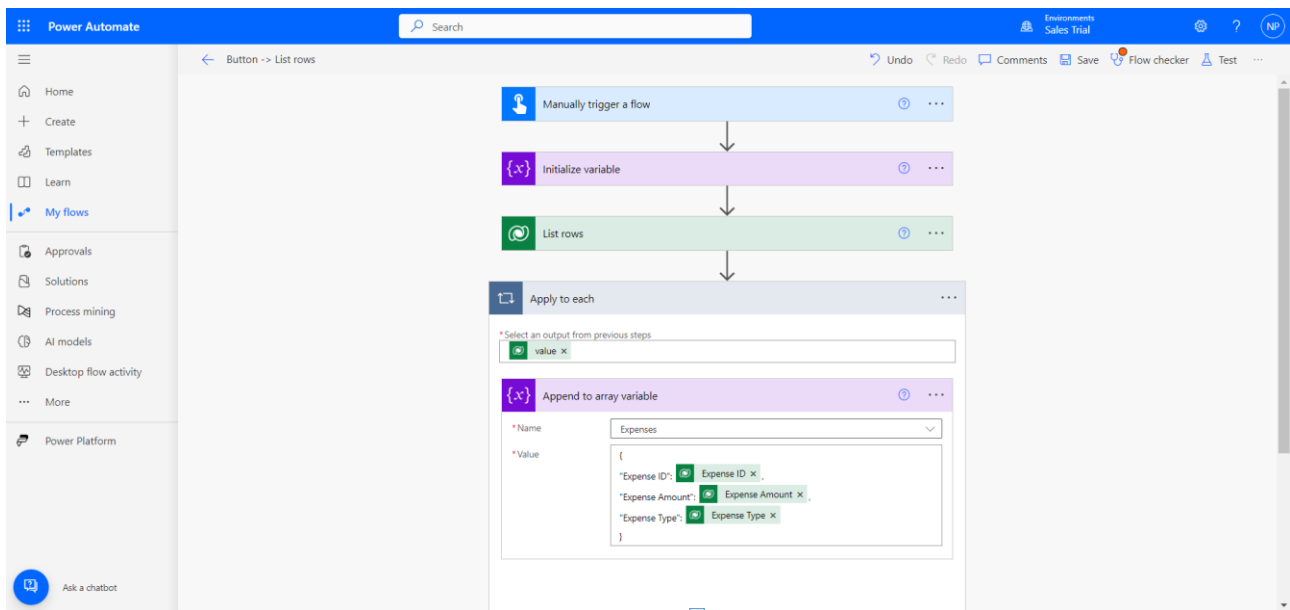
To actually see the data we collect in the Expenses array variable, we use a Compose step. Then we obtain the result in the following figure.



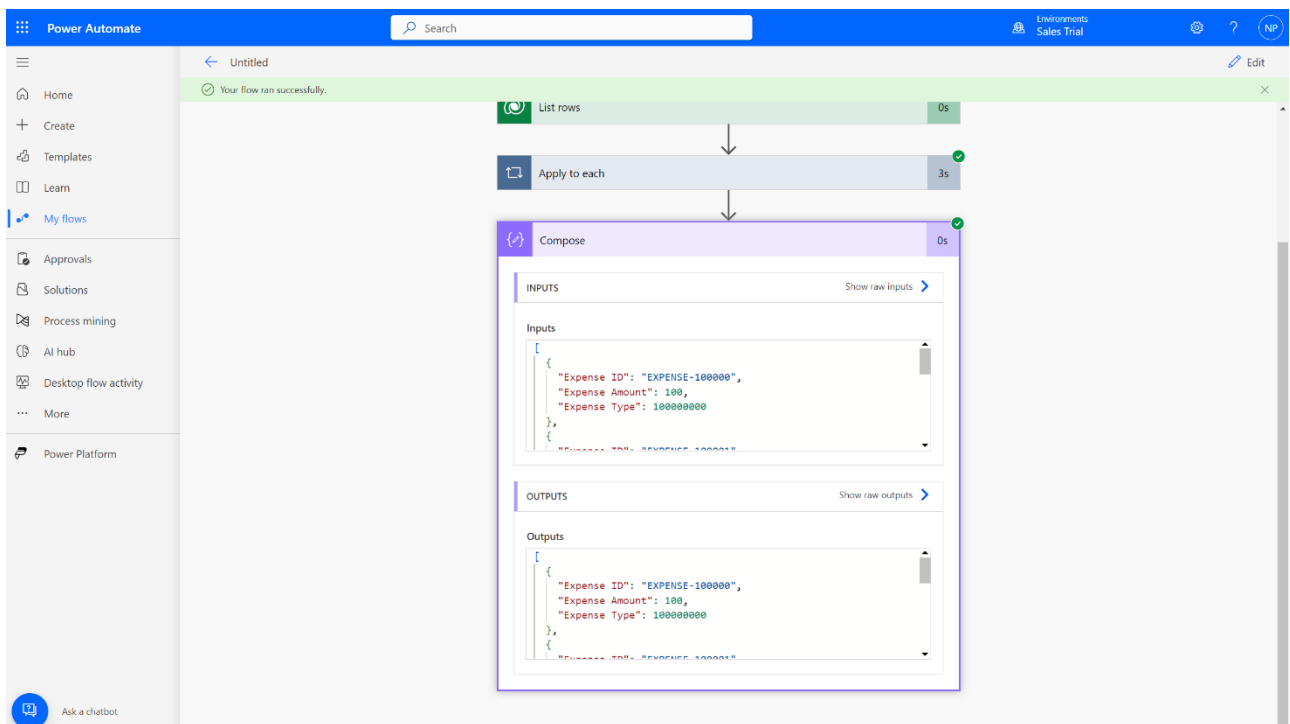
Instead of this format we may want something as:

```
[
  {
    "ID": "EXPENSE-100000",
    "Amount": "500",
    "Expense Type": "Flight"
  },
  {
    "ID": "EXPENSE-100001",
    "Amount": "600",
    "Expense Type": "Food"
  }
]
```

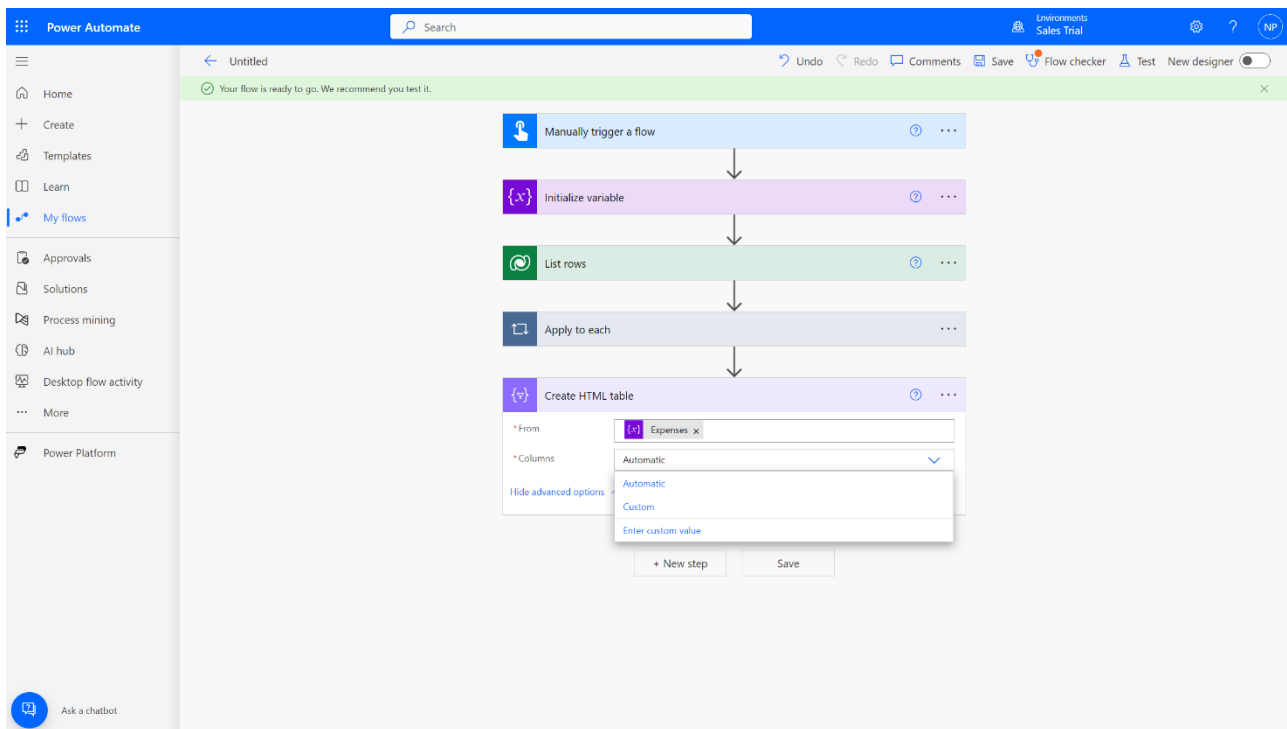
To achieve this, we modify the Append to array variable step as follows.



The output will be the following.

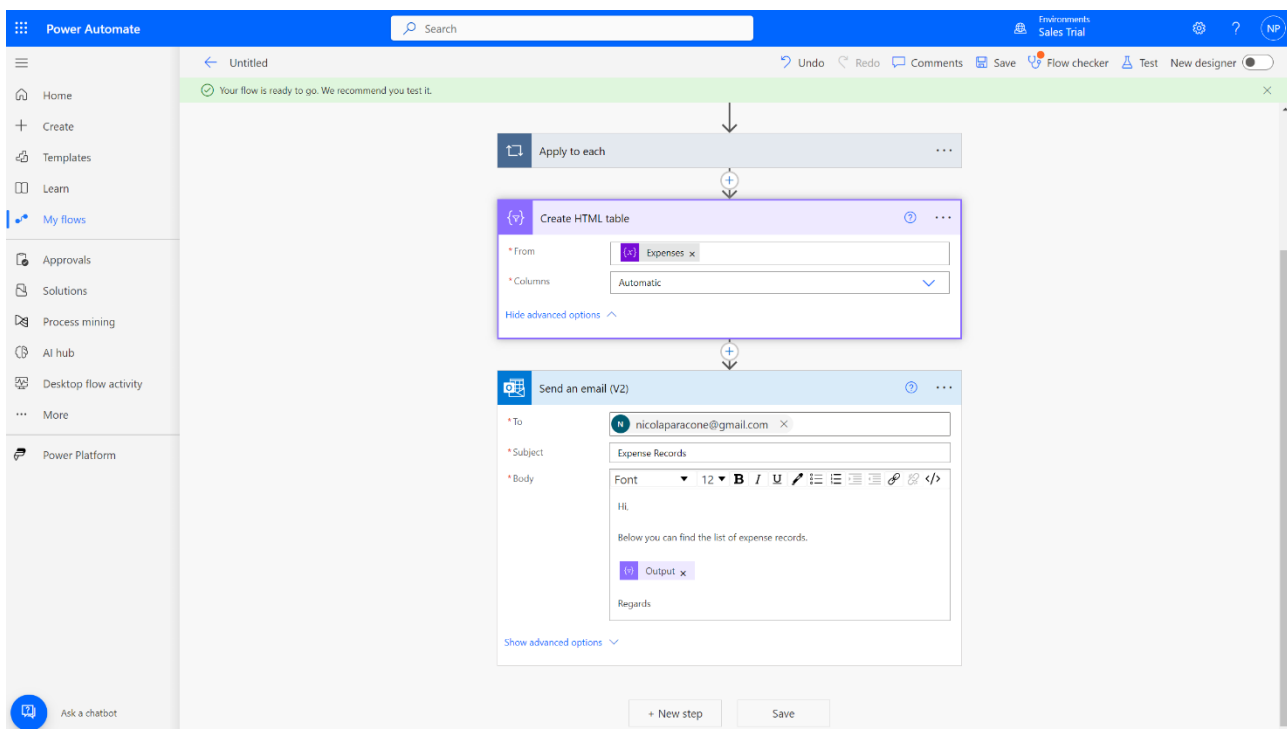


At this point we create a table out of it. Delete the Compose block, add a new step and search for Data Operation -> Create HTML table.



In choosing the columns we can do it automatically or specify the keys resulting from the previous step (each of them defines a column).

Finally, we add a send email block.



The email we will send has the following appearance.

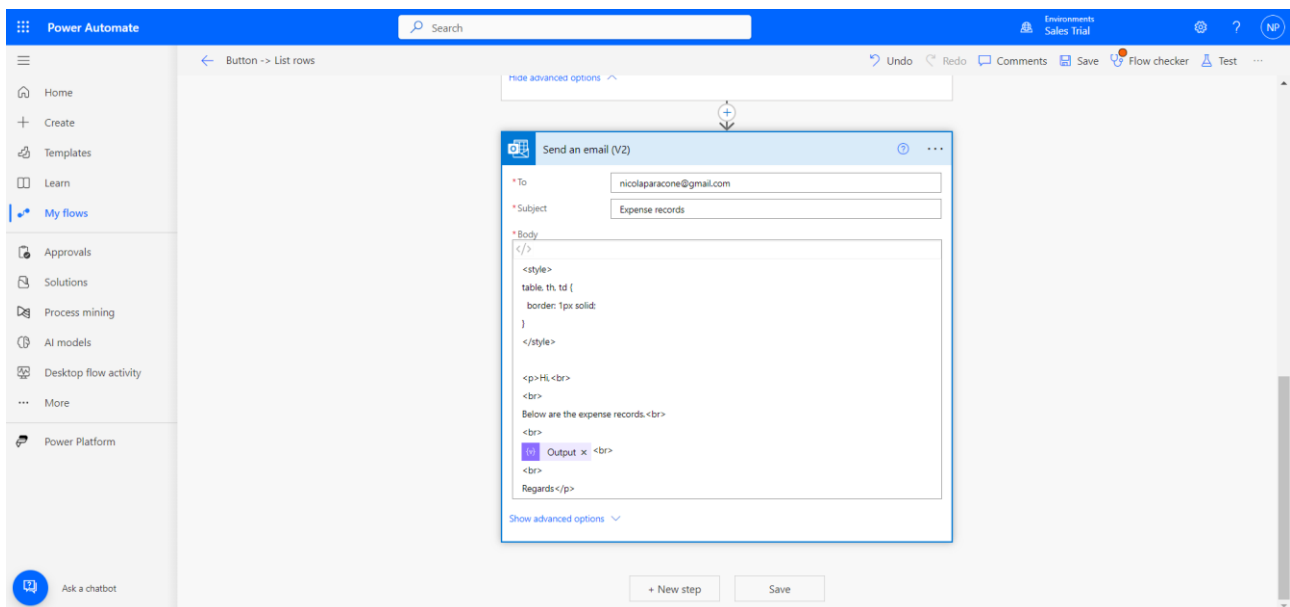
Hi,

Below you can find the list of expense records.

Expense ID	Expense Amount	Expense Type
EXPENSE-100000	100	100000000
EXPENSE-100001	600	100000002
EXPENSE-100002	200	100000000
EXPENSE-100003	300	100000000
EXPENSE-100004	2000	100000003

Regards

Of course, we can format the table by defining our own CSS in the body of the email.



The new structure of the email will be as in the following figure.

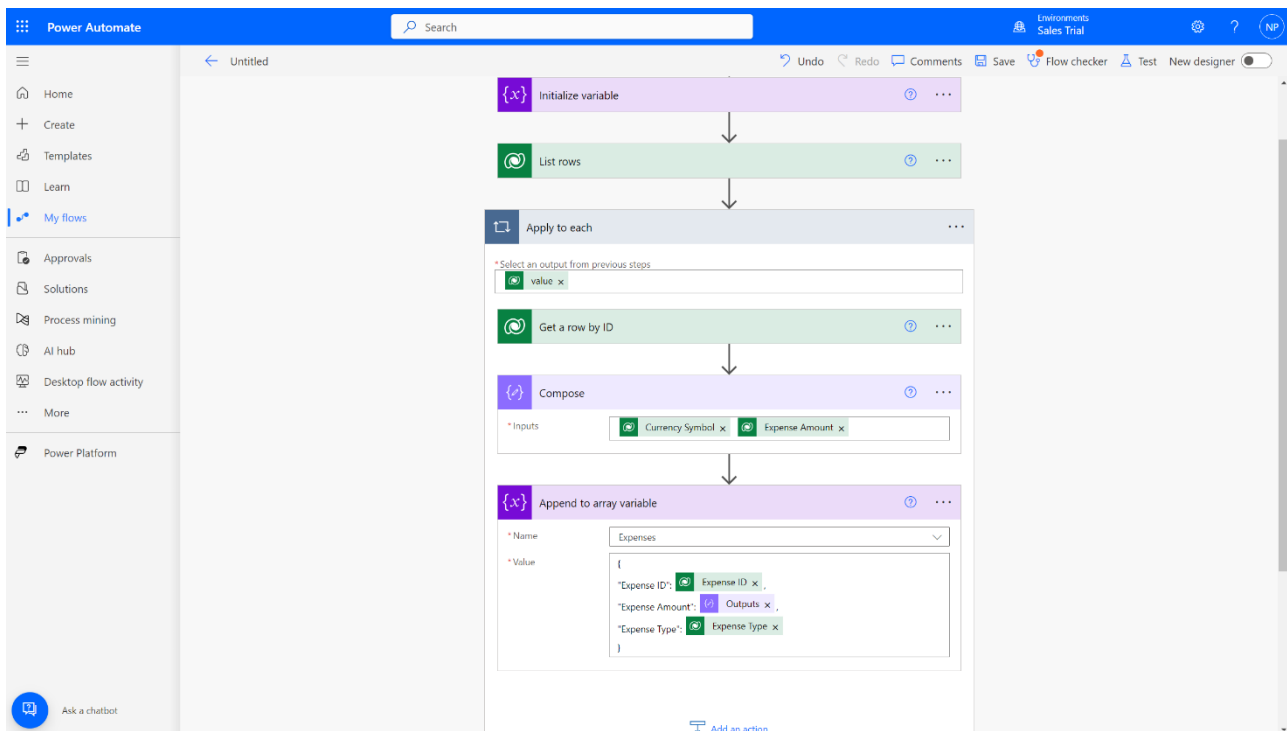
Hi,

Below you can find the list of expense records.

Expense ID	Expense Amount	Expense Type
EXPENSE-100000	100	100000000
EXPENSE-100001	600	100000002
EXPENSE-100002	200	100000000
EXPENSE-100003	300	100000000
EXPENSE-100004	2000	100000003

Regards

If we want to retrieve also the currency, we modify the Apply to each block as follows.



The email will be the following.

Hi,

Below you can find the list of expense records.

Expense ID	Expense Amount	Expense Type
EXPENSE-100000	€ 100	100000000
EXPENSE-100001	€ 600	100000002
EXPENSE-100002	€ 200	100000000
EXPENSE-100003	€ 300	100000000
EXPENSE-100004	€ 2000	100000003

Regards

Alternatively, we can use the concat expression:

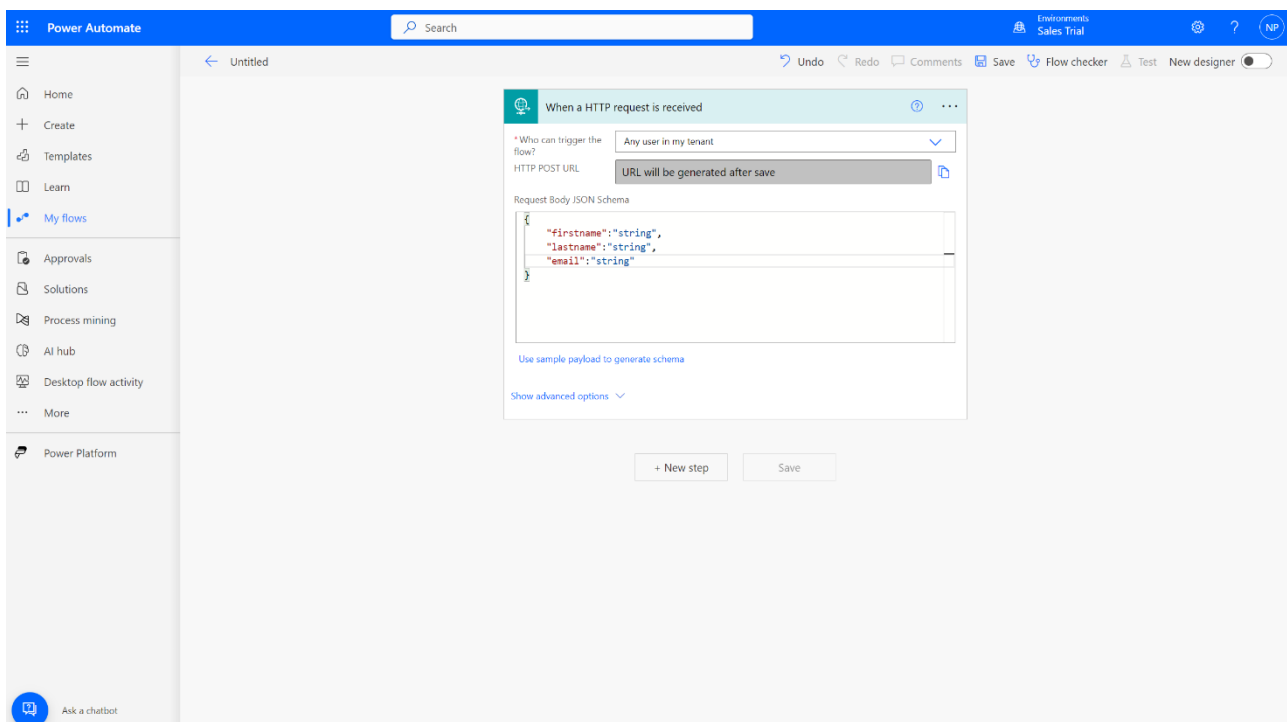
```
concat(outputs('Get_a_row_by_ID')?['body/currencysymbol'],items('Apply_to_each')?['demo_expenseamount'])
```

4. Flows triggered by an HTTP request

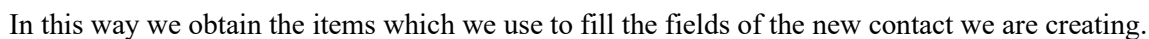
Let us create an automated cloud flow triggered by an HTTP request. HTTP, or Hypertext Transfer Protocol, serves as the fundamental protocol for data communication on the World Wide Web. It operates at the application layer and facilitates the exchange of information between a web server and a client, typically a web browser. HTTP is employed for the retrieval and submission of various web resources, including HTML pages, images, and other file types. In the HTTP model, there is a client-server arrangement where the client, often a web browser, sends requests for resources, and the server responds accordingly. This communication takes place over the Internet using the TCP/IP (Transmission Control Protocol/Internet Protocol) suite. HTTP possesses certain characteristics, such as being a stateless protocol, treating each client-server interaction as an independent transaction. It is also connectionless, allowing for the closure of connections after a request is made and a response is provided. Additionally, HTTP messages are in plain text, ensuring human readability and facilitating debugging. The protocol follows a request-response model, wherein clients initiate requests, and servers reply with the requested information or an error message if applicable. While HTTP/1.1 has been the prevalent version for an extended period, there has been a recent shift toward adopting newer versions like HTTP/2 and HTTP/3. These versions aim to enhance performance by introducing features such as multiplexing, header compression, and other optimizations, addressing limitations present in earlier versions.

In the context of Power Automate and Dynamics 365, a HTTP block is used, for instance, when we want to do some calculations or create any record in our database from an external source. The When a HTTP request is received in Power Automate serves as a trigger for initiating workflows in response to incoming HTTP requests. This action allows Power Automate to listen for and respond to requests sent by external systems or applications. Users can define an endpoint URL, and when an HTTP request is made to this URL, the workflow is triggered. The action is designed to handle the payload or data that may be included in the HTTP request, enabling users to extract and process this information as required. It essentially provides a way to integrate and automate workflows based on external events signaled through HTTP requests.

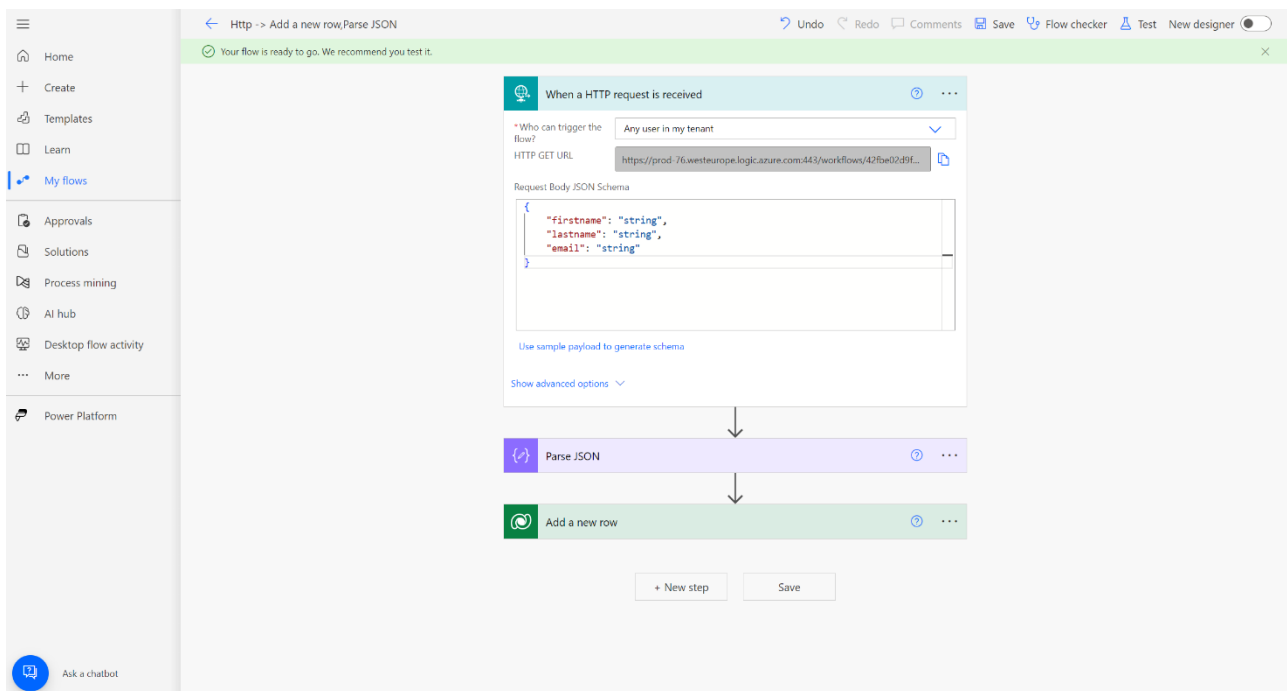
When first adding the When a HTTP request is received trigger to a flow, we are presented with a HTTP POST URL informing us that the URL will be generated after the flow has been saved. This means that while we are initially creating the flow, we will not be able to provide/use the URL to that is required to trigger the flow.



So, what do we want to do when we receive those data? For example, we can create a new contact. First, we need to read the JSON using a Parse JSON block. Here we click on Generate from sample and copy and paste our JSON schema. The process of parsing involves taking the JSON and converting it into a data structure that a programming language or tool can understand. Once parsed, the JSON data becomes a native data structure in the programming language. For example, in many languages, it might become an object, a dictionary, or a map.

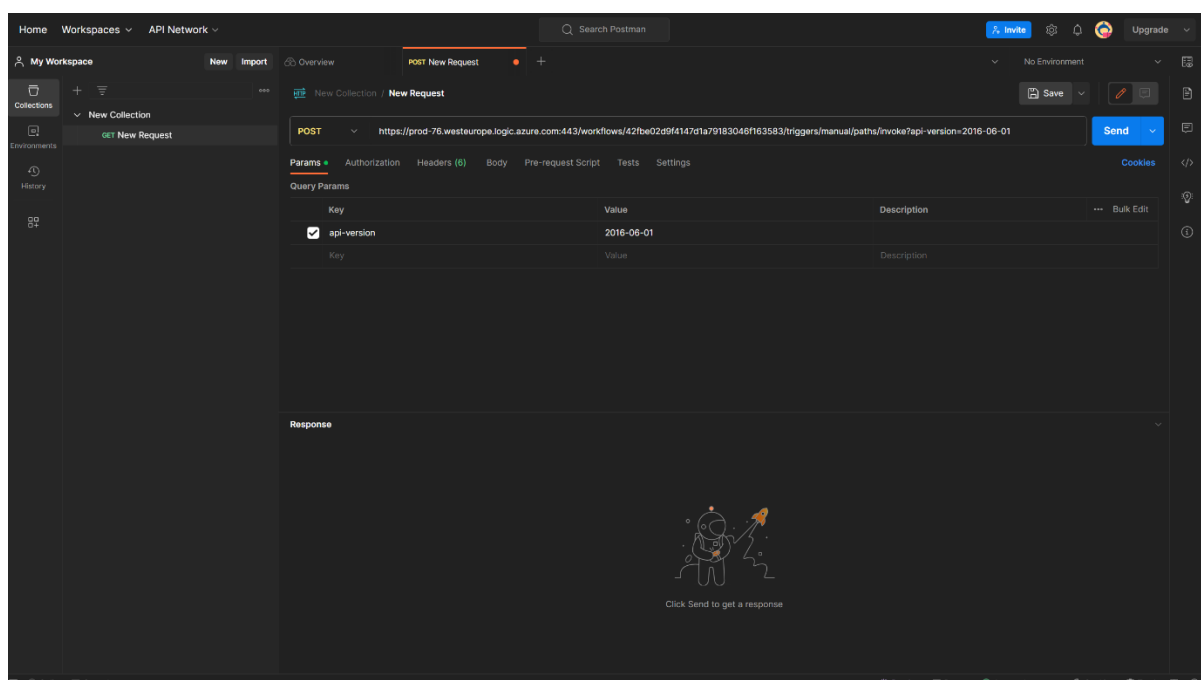


Once we are done setting up the fields, after saving we will have the URL being generated.

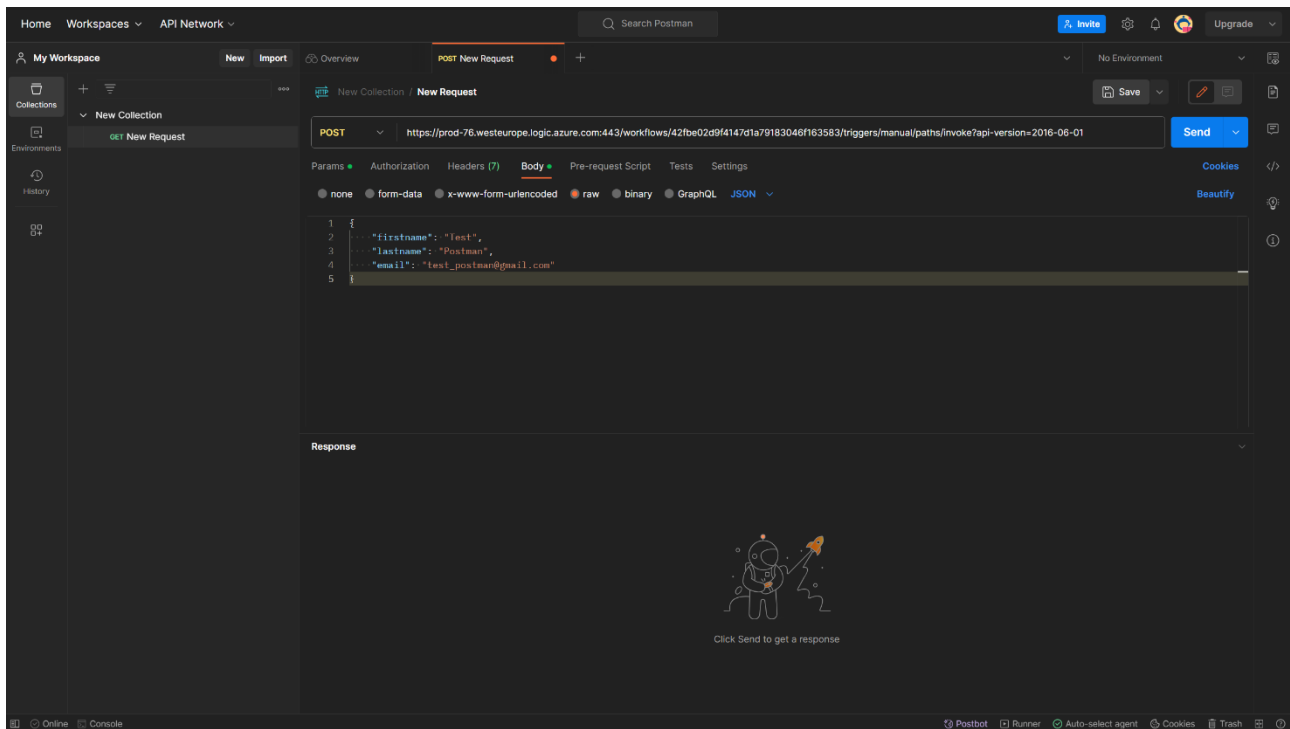


Copy the URL and navigate to <https://www.postman.com>. Postman is a collaborative platform designed for the development and testing of APIs. It provides a user-friendly interface that assists developers in creating, testing, sharing, and documenting APIs. With features such as organized collections, environment variables, and automation capabilities, Postman simplifies the process of working with APIs. It supports various HTTP methods, allowing users to execute requests, inspect responses, and debug APIs. Additionally, Postman offers tools for creating mock servers, monitoring API performance, and generating interactive documentation. The platform promotes collaboration by enabling teams to share collections, workspaces, and documentation. Postman is available as both a web and desktop application and is widely used across the development community.

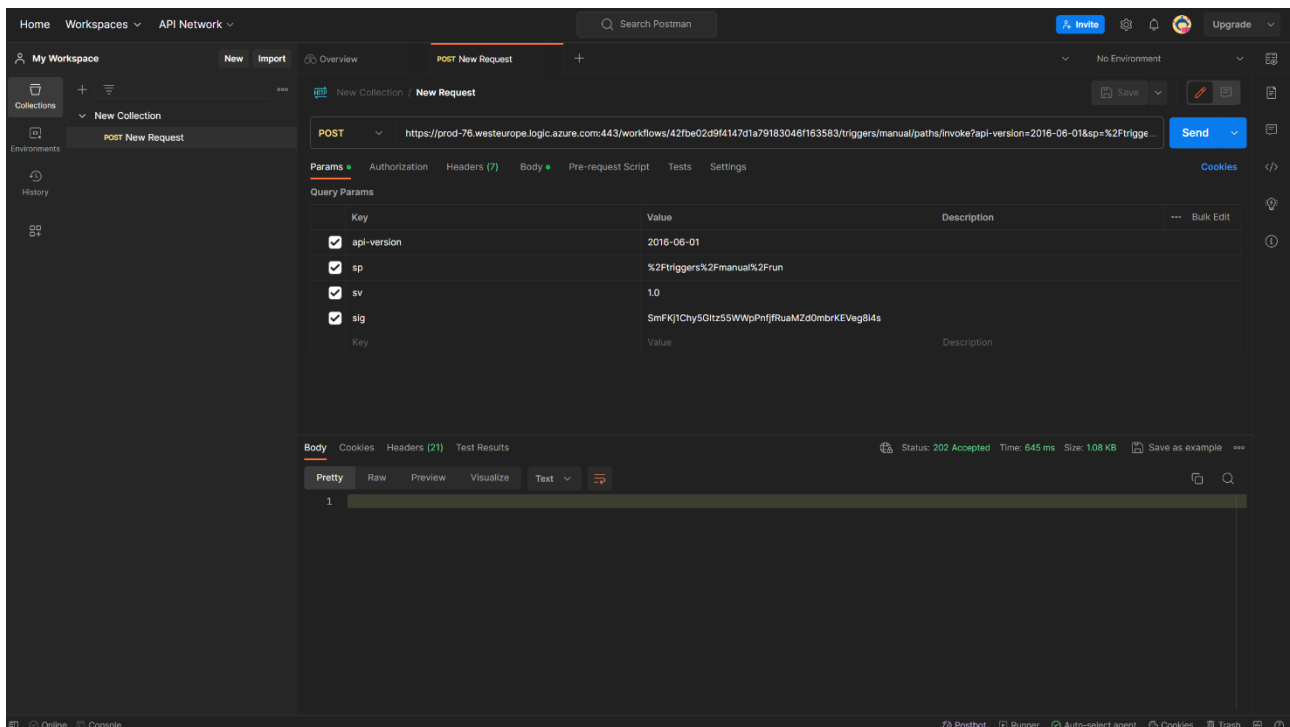
Once logged in, create a new request, set the method to POST and paste the copied URL.



Copy the JSON schema you defined previously and paste it into Body -> raw on Postman (modify the values of the keys as you wish). Set Authorization to No Auth (usually it will be OAuth 2.0 when we try to connect to Dynamics 365).



Click on Send.



If an error message appears you can try to modify the field Who can trigger the flow? inside the When a HTTP request is received block (maybe set it to Anyone).

In our case the flow ran successfully.

The screenshot displays the Microsoft Power Automate web interface. At the top, a green notification bar states: "Your flow is ready to go. It's turned on and will run when it's triggered." The main area shows the details of a flow named "Http -> Add a new row, Parse JSON".

Flow Details:

- Flow: Http -> Add a new row, Parse JSON
- Status: On
- Owner: Nicola Paracone
- Created: Dec 28, 12:07 PM
- Modified: Dec 28, 12:24 PM
- Type: Automated
- Plan: This flow runs on owner's plan

28-day run history:

Start	Duration	Status
Dec 28, 12:24 PM (1 min ago)	00:00:05	Succeeded

Connections: Microsoft Dataverse | NParacone@hotelnet.tech

Owners: NP Nicola Paracone

Process mining (preview): Average run duration: 00:00:05

Associated Apps: You don't have any apps associated with this flow. [Learn more](#)

Opening Contacts in the Expense System app, we will notice that a new contact has appeared.

The screenshot shows the Microsoft Dynamics 365 Expense System interface. The left sidebar includes navigation options: Home, Recent, Pinned, Expense Group, Contacts, Expenses, and Open Google. The main area displays "My Active Contacts" with a table of contact information.

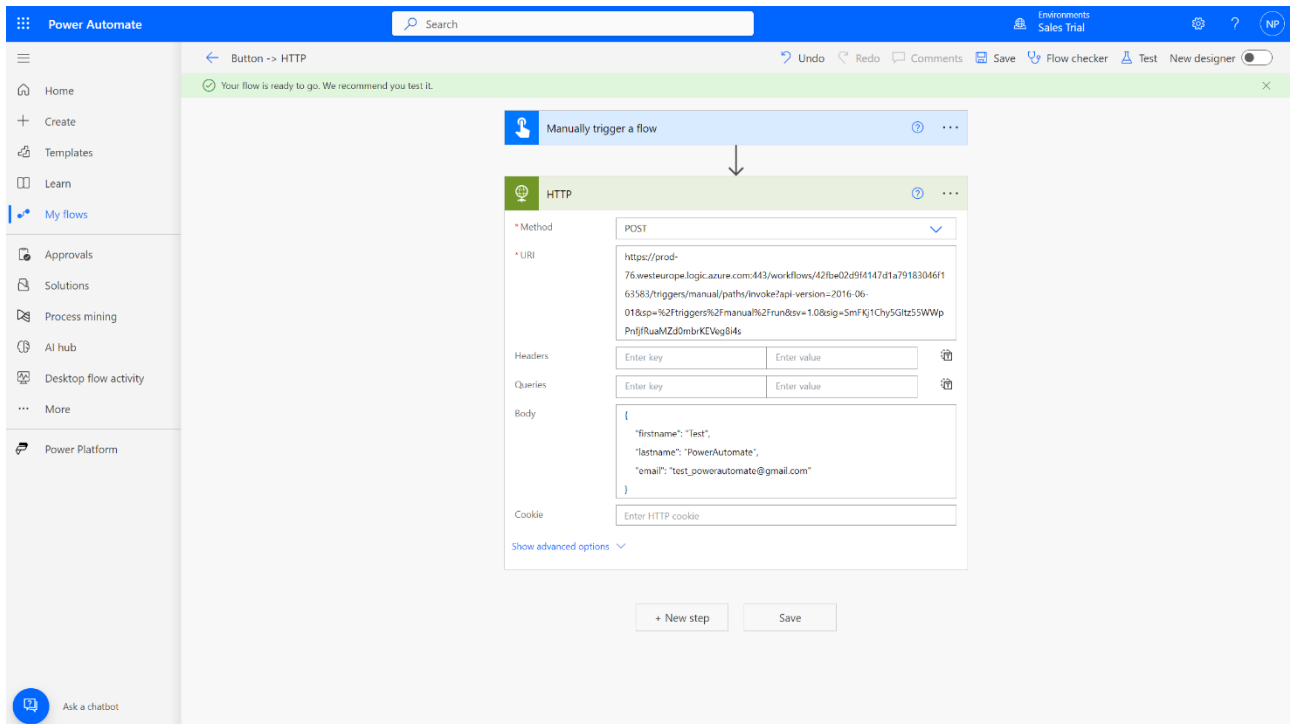
My Active Contacts

Full Name	Email	Company Name	Business Phone
Alex Baker	alex@treyresearch.net	Trey Research	619-555-0127
Avery Howard	nicolaparacone@gmail.com	Trey Research	567-555-0137
Cacilia Viera	cacilia@alpineskihouse.com	Alpine Ski House	281-555-0162
Carla Yates	carla@treyresearch.net	Trey Research	456-555-0156
Dwayne Elijah	dwayne@alpineskihouse.com	Alpine Ski House	281-555-0160
Haroun Stormonth	haroun@fabrikaminc.com	Fabrikam, Inc.	423-555-0122
Henberto Nathan	henberto@northwindtraders.com	Northwind Traders	614-555-0123
Kenya Brady	kenya@consolidatedmessenger.com		222-555-0172
Kevin Martin	kevin@adatum.com	A. Datum Corporation	425-555-0160
Kim Rocha	kim@treyresearch.net	Trey Research	567-555-0157
Miguel Garcia	miguel@northwindtraders.com	Northwind Traders	614-555-0127
Test Postman	test_postman@gmail.com		
Zoltán Szabó	zoltan@fabrikaminc.com	Fabrikam, Inc.	423-555-0120

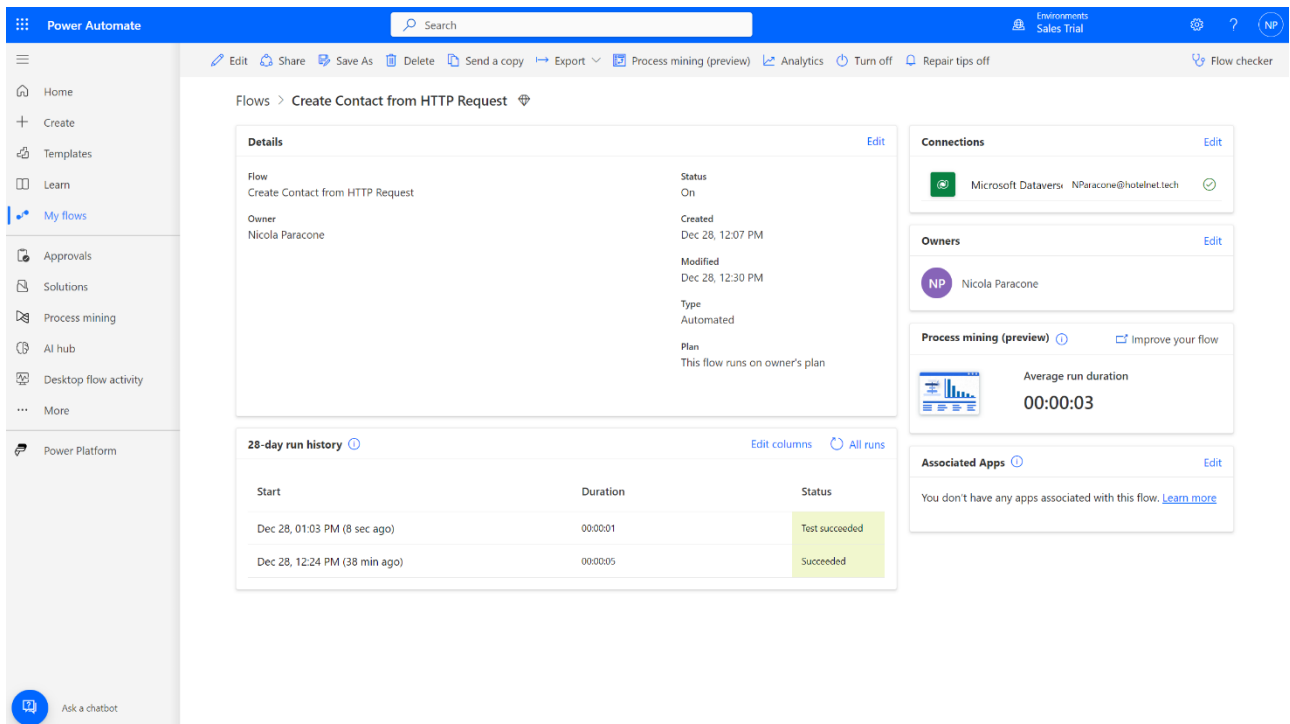
1 - 13 of 13 (1 Selected) Page 1

5. Call an API

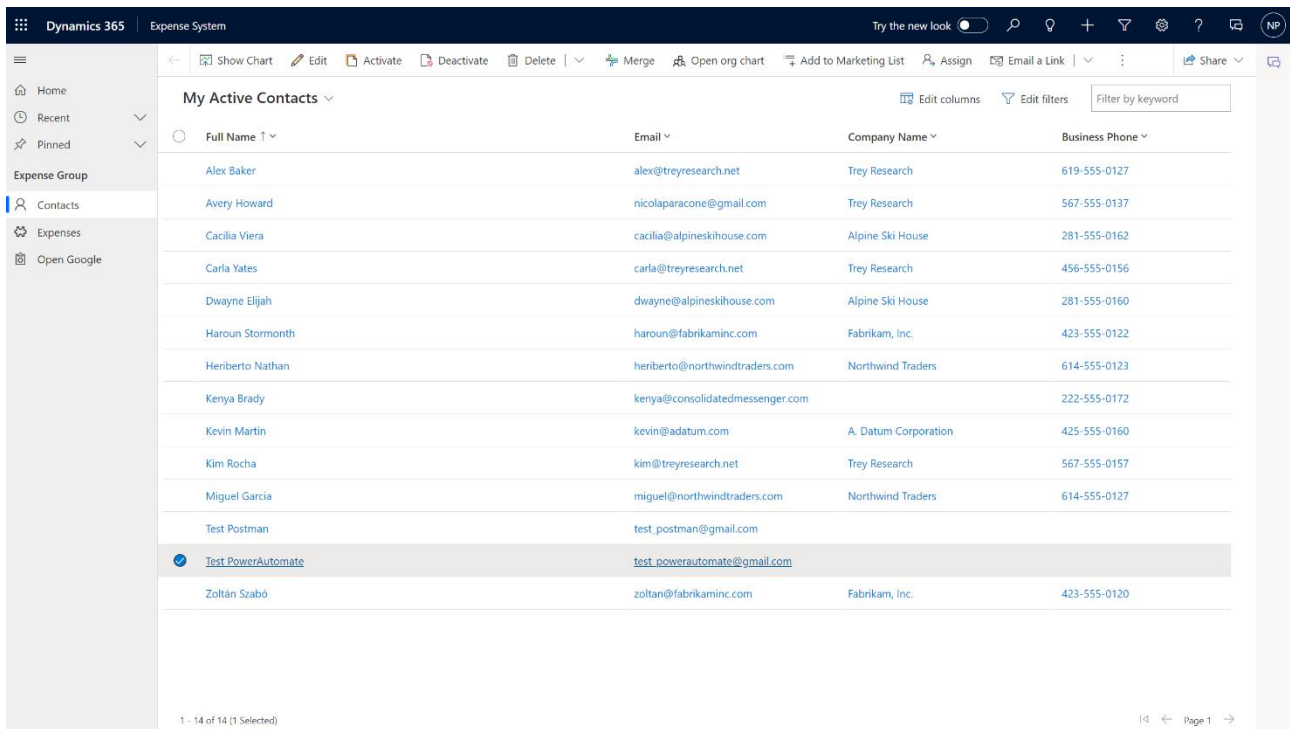
Now we learn how to call an API; basically, we will implement in Power Automate the operations we have done in Postman. We create a new automated cloud flow and choose Manually trigger a flow as trigger. At the next step we add a HTTP block, with method POST and as URL the one that we pasted into Postman. At the next step we add a HTTP block, with method POST and as URL the one that we pasted into Postman.



Once the flow has been saved and we have clicked on Test, we check the history of the previous flow and see that it has been correctly triggered.



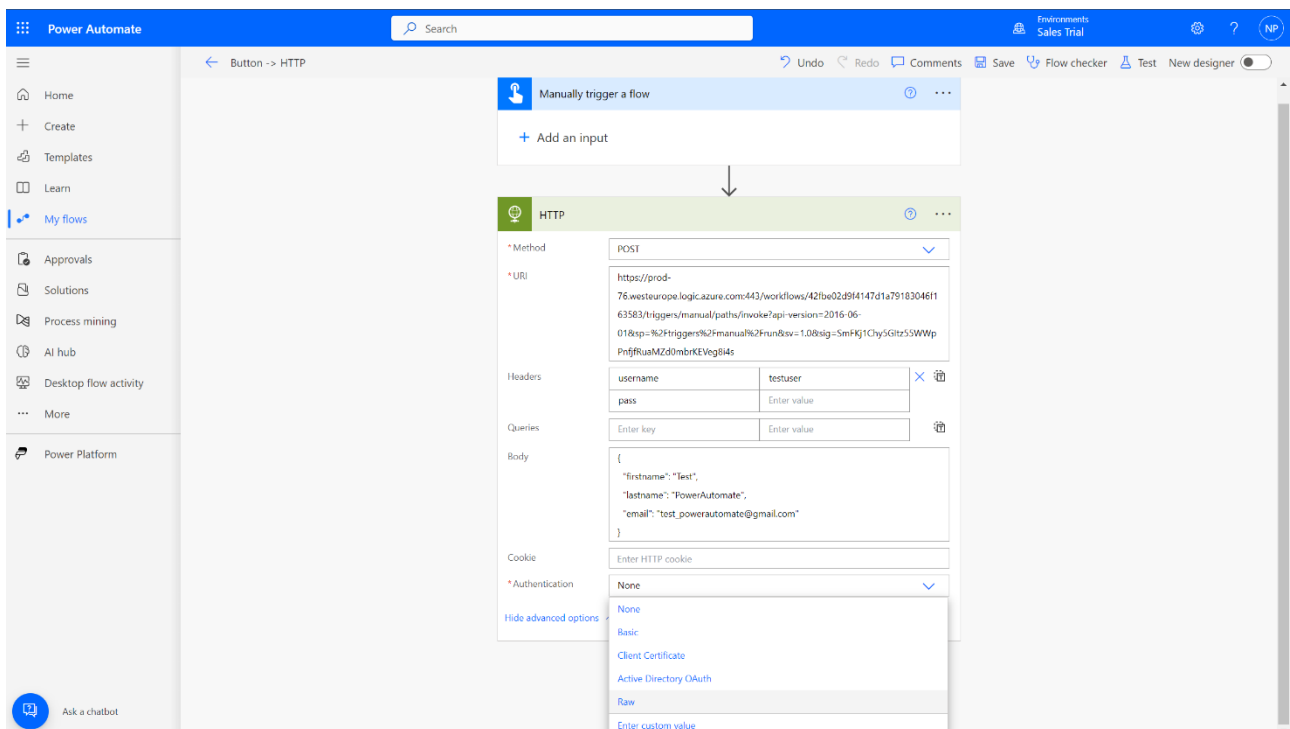
A new contact appears in Dynamics 365.



The screenshot shows the Dynamics 365 interface with the 'My Active Contacts' list. The list includes columns for Full Name, Email, Company Name, and Business Phone. The contact 'Test PowerAutomate' is selected, indicated by a blue checkmark in the selection column.

Full Name	Email	Company Name	Business Phone
Alex Baker	alex@tresearch.net	Trey Research	619-555-0127
Avery Howard	nicolaparacone@gmail.com	Trey Research	567-555-0137
Cacilia Viera	cacilia@alpineskihouse.com	Alpine Ski House	281-555-0162
Carla Yates	carla@tresearch.net	Trey Research	456-555-0156
Dwayne Elijah	dwayne@alpineskihouse.com	Alpine Ski House	281-555-0160
Haroun Stormonth	haroun@fabrikaminc.com	Fabrikam, Inc.	423-555-0122
Heriberto Nathan	heriberto@northwindtraders.com	Northwind Traders	614-555-0123
Kenya Brady	kenya@consolidatedmessenger.com		222-555-0172
Kevin Martin	kevin@adatum.com	A. Datum Corporation	425-555-0160
Kim Rocha	kim@tresearch.net	Trey Research	567-555-0157
Miguel Garcia	miguel@northwindtraders.com	Northwind Traders	614-555-0127
Test Postman	test_postman@gmail.com		
Test PowerAutomate	test_powerautomate@gmail.com		
Zoltán Szabó	zoltan@fabrikaminc.com	Fabrikam, Inc.	423-555-0120

More fields can be added to the HTTP block.



The screenshot shows the Power Automate interface with an HTTP action configured. The 'Add an input' button is visible above the HTTP block. The configuration includes a POST method, a specific URL, headers for username and password, and a JSON body with first name, last name, and email fields.

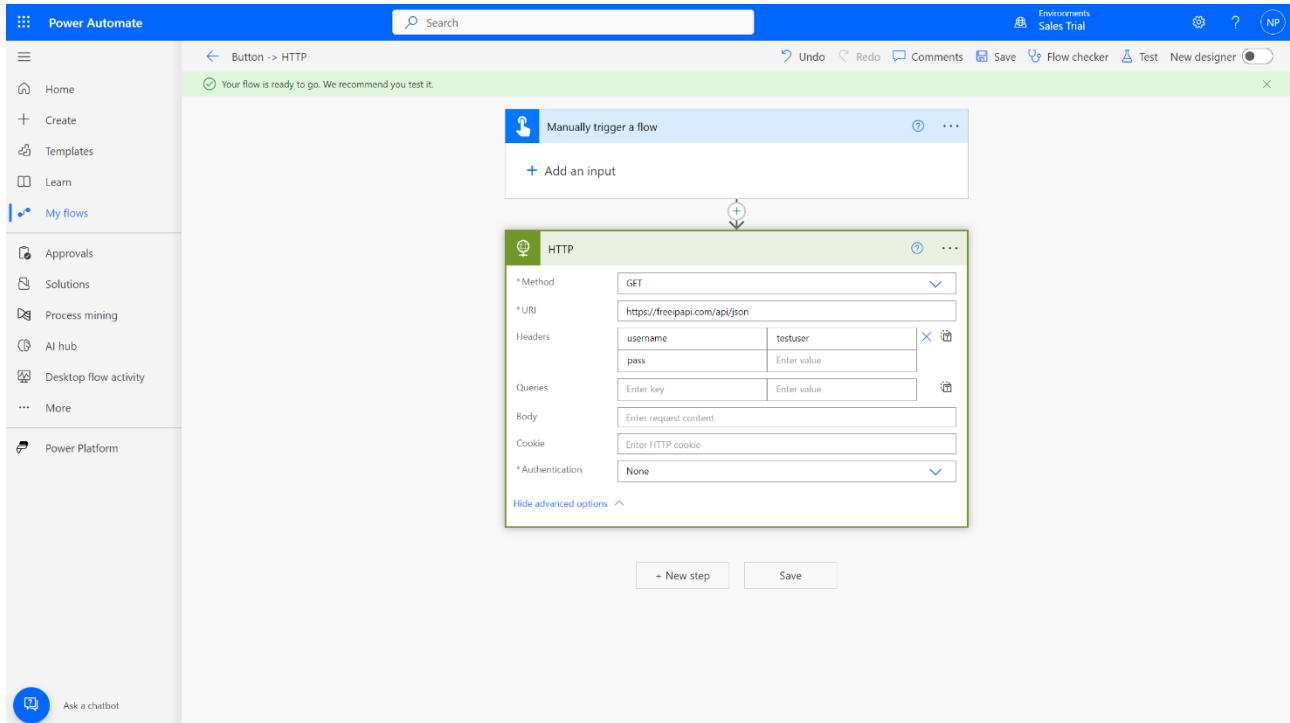
HTTP Action Configuration:

- Method:** POST
- URI:** https://prod-76westeurope.logic.azure.com:443/workflows/42fbc02d9f4147d1a79183046f163583/triggers/manual?api-version=2016-06-01&sp=%2Ftriggers%2Fmanual%2Frun&sv=1.0&sig=5mFKJ1Chy5Gltz5WwPpNfjRuaMZd0mbrKEVeg84s
- Headers:**
 - username: testuser
 - pass: Enter value
- Queries:**
 - Enter key: Enter value
- Body:**

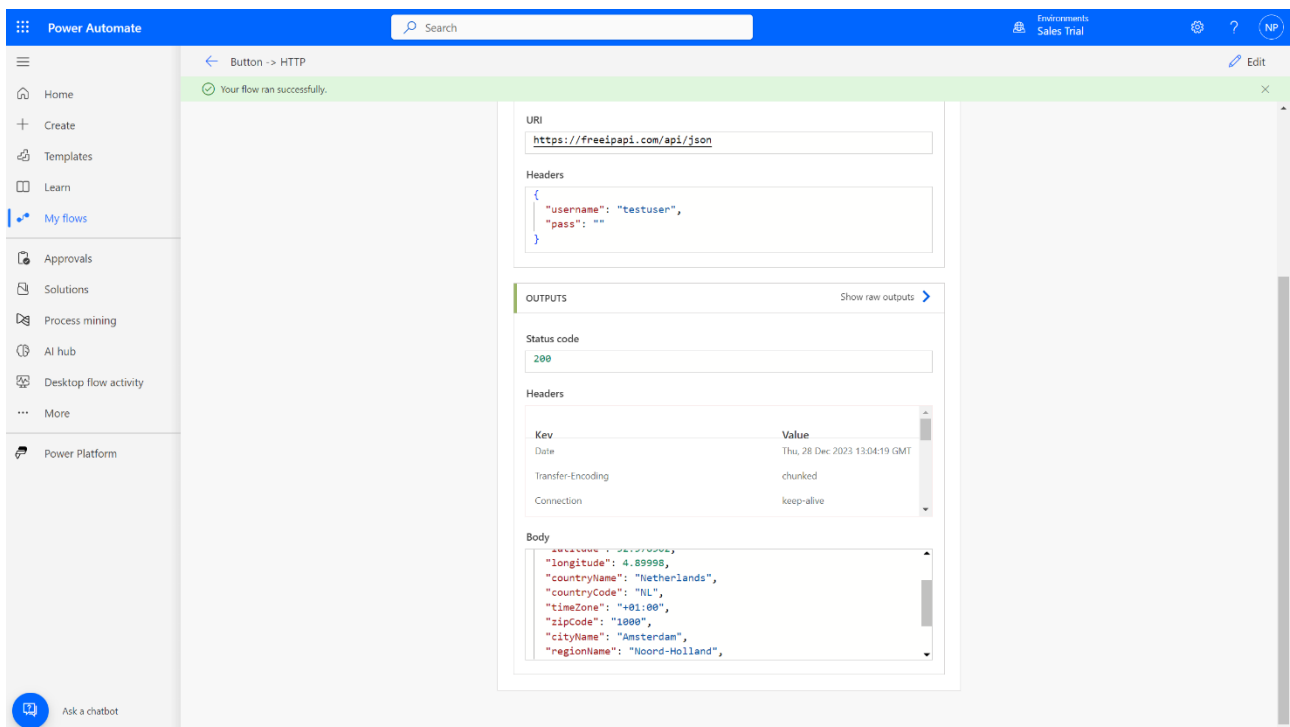
```
{  "firstname": "test",  "lastname": "PowerAutomate",  "email": "test_powerautomate@gmail.com"}
```
- Cookie:** Enter HTTP cookie
- Authentication:** None

At this point we learn how to call an external API by means of the same HTTP block. In this case we want to get, not to post.

To perform this test, we look for a sample API in the web. For instance, navigate to <https://freeipapi.com>, which is a geolocation API.

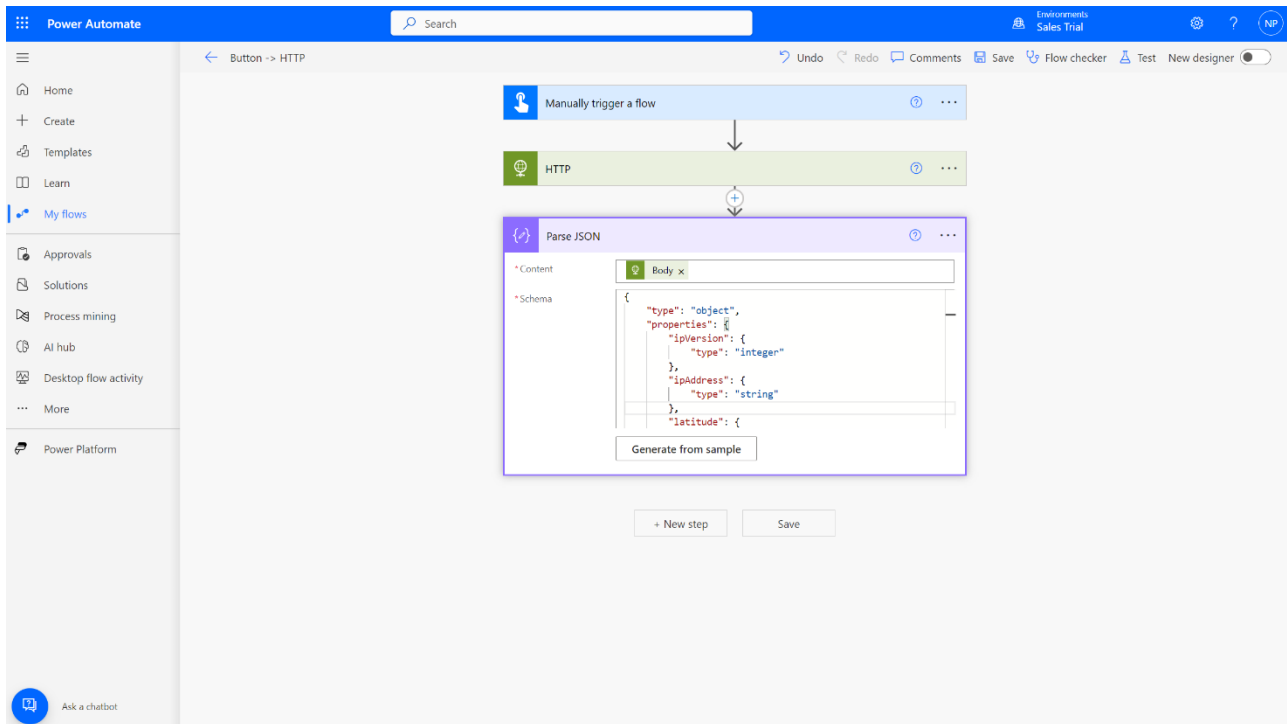


As output, we obtain the following.

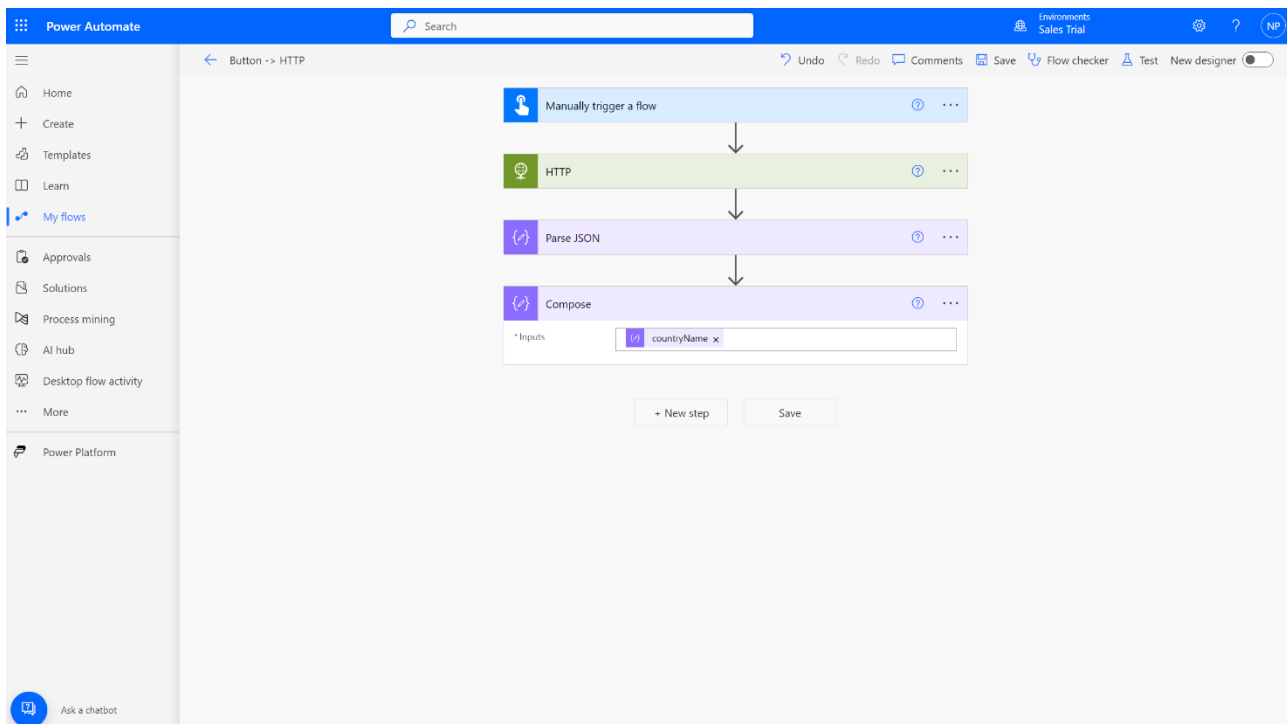


Next, we copy the body we received and add a new Parse JSON block.

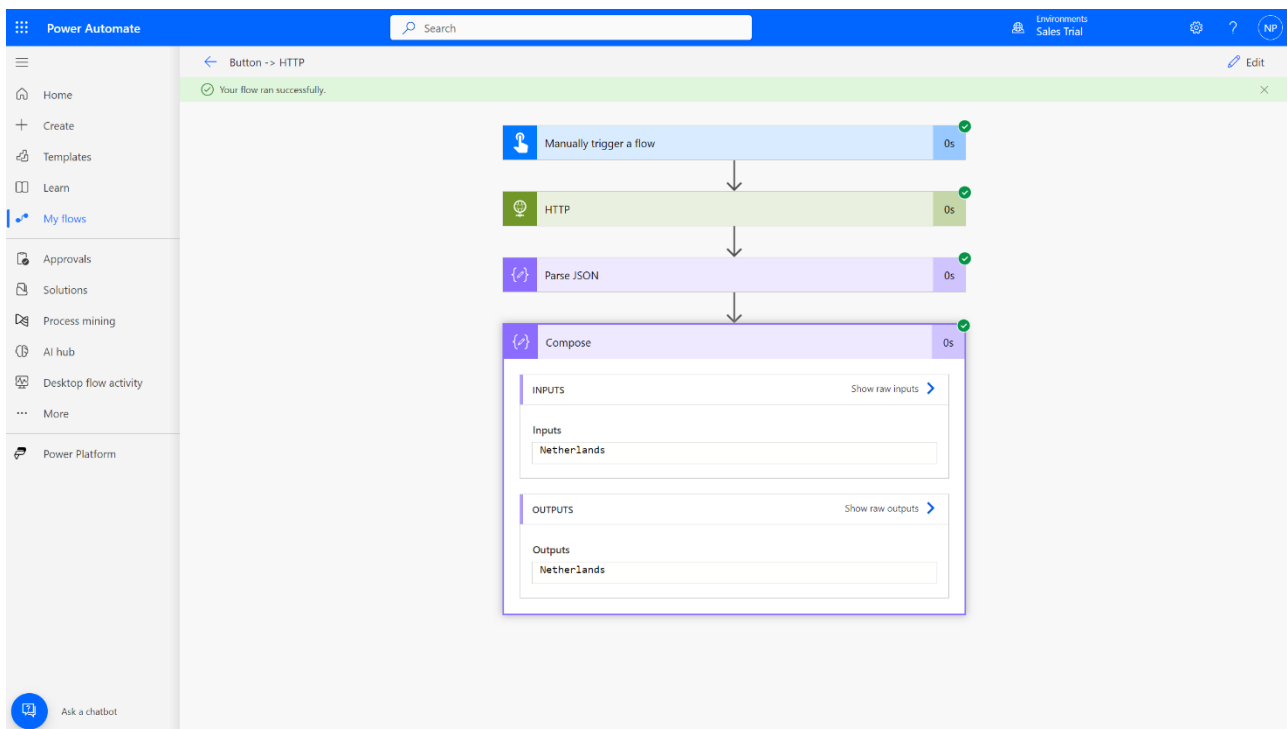
Use Generate from sample to create the Schema (copy and paste the JSON obtained as output in the previous test).



Add a Compose block to read just countryName.

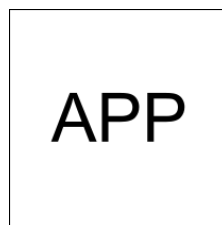


Save and test.



6. General introduction to APIs

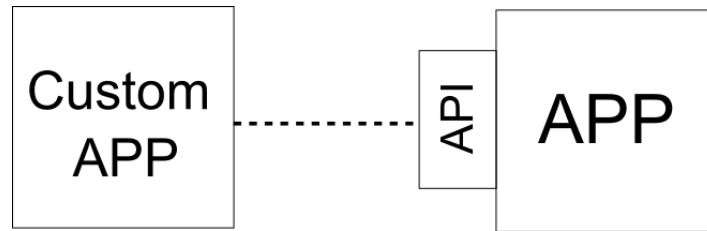
API stands for Application Programming Interface. Suppose that we have built an application, which has taken us years of development and a lot of money.



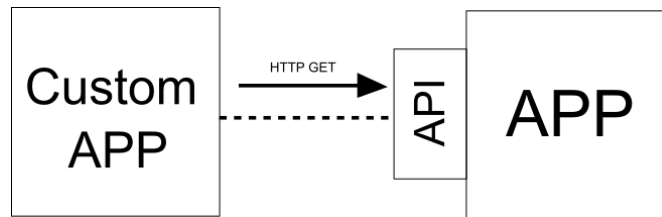
If we want this application to be a worldwide success, we need to integrate it with other popular applications and services. This is where APIs come in. We can build an API onto our application, which is essentially a doorway into our application.



Using this doorway, other third-party applications can get access to all of the data and features we built without us needing to provide any custom code whatsoever.

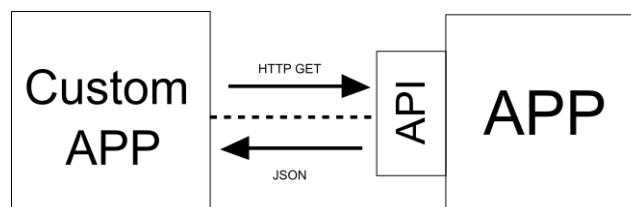


Let's say that the custom app wants to access some of the data or features that we have built. It will send an API request. API requests are sent in the form of a simple HTTP message.

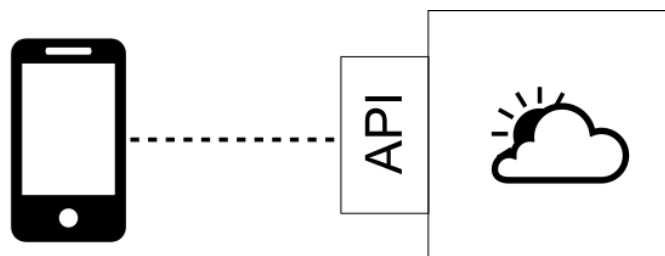


In this example, we are using a GET message to request information. There are other HTTP methods that we could use: POST, GET, PUT, PATCH, DELETE. These methods correspond to something called CRUD operations: CREATE, READ, UPDATE, UPDATE, DELETE. The POST method creates something on the web application, the GET method requests read information from the application, PUT updates information of the application, PATCH also updates information on the application (PUT will replace the information, while PATCH will modify it), DELETE deletes the information.

Once the API request has been sent, the application will send back a response. The response will be sent back to the custom app usually in a JSON or XML format.



Suppose now that the app is a weather app, and that the custom app is a mobile.



Building a worldwide network of weather stations and compiling all of that information is nearly impossible. But this is the beauty of APIs. Anyone can build a mobile app and then use APIs to request the data from an already existing weather application. The mobile app sends an API GET request to the weather application.

The following is an example of an API request using the openweathermap API:

`https://api.openweathermap.org/data/2.5/weather?q={city name}&appid={API key}`

It is a simple URL which contains some field. One field is where the app would enter the city, while the other field is an API key. An API key is usually required so that the application can keep track of the number of requests being sent and also make sure that we are not spamming them. Once the application receives this request, it will respond with the requested data. The following is a small snippet of one of the responses:

```
"main":{  
  "temp": 9.53,  
  "feels_like": 7.62,  
  "temp_min": 7.78,  
  "temp_max": 10.56,  
  "pressure": 1016,  
  "humidity": 61  
}
```

This is in JSON format. We can see different names and values here, and we can program our mobile app to take this information and display it in a nice user interface.